

Ideias para Plataforma de Torneios osu!

<https://gemini.google.com/app/1c19a1e16b1e44d1>

User prompt: Estou pensando em criar um site pra ajudar minha comunidade de osu envolvendo torneios. Sou um estudante de desenvolvimento sênior e já criei aplicações anteriormente envolvendo a API do jogo. Consegue me dar algumas ideias? --- url: <https://osu.ppy.sh/community/matches/121918084> title: "(1) 6WC26'ML: (Brazil B) vs (United States B) · partidas | osu!" date: 2026-09-26T11:36:08.081Z lang: pt-BR --- [(https://osu.ppy.sh/partidas Toggle navigation [Kr3Mega](https://osu.ppy.sh/users/21496977) [0] 26T11:36:08.081Z lang: pt-BR --- [(https://osu.ppy.sh/community/chat) [1](https://osu.ppy.sh/notifications) [! [4022afa23b3ac8bd519a2a0eea71b1c27bc87e5d00690c97de6258106026baf8](https://assets.ppy.sh/user-profile-covers/21496977/4022afa23b3ac8bd519a2a0eea71b1c27bc87e5d00690c97de6258106026baf8.png)](https://osu.ppy.sh/users/21496977) ! [21496977?1781834950](https://a.ppy.sh/21496977?1781834950.png) [(https://osu.ppy.sh/rankings/osu/performance?country=BR) [Kr3Mega] (https://osu.ppy.sh/users/21496977) Online Modo Lazer [Meu Perfil](https://osu.ppy.sh/users/21496977) [Criar equipe] (https://osu.ppy.sh/teams/create) [Amigos](https://osu.ppy.sh/home/friends) [Listas de observação](https://osu.ppy.sh/home/follows/forum_topic [Configurações](https://osu.ppy.sh/home/account/edit) Desconectar [início](https://osu.ppy.sh) * [notícias](https://osu.ppy.sh/home/news) * [equipe](https://osu.ppy.sh/wiki/pt-br/People/osu%21_team) * [registro de alterações](https://osu.ppy.sh/home/changelog) * [baixar] (https://osu.ppy.sh/home/download) * [pesquisar](https://osu.ppy.sh/home/search) [beatmaps](https://osu.ppy.sh/beatmapsets) * [lista de beatmaps](https://osu.ppy.sh/beatmapsets) * [artistas em destaque](https://osu.ppy.sh/beatmaps/artists) * [pacotes de beatmap] (https://osu.ppy.sh/beatmaps/packs) [colocações](https://osu.ppy.sh/rankings/osu/global) * [global](https://osu.ppy.sh/rankings/osu/global) * [país](https://osu.ppy.sh/rankings/osu/country) * [recordes](https://osu.ppy.sh/rankings/top-plays/osu) * [equipe] (https://osu.ppy.sh/rankings/osu/team) * [playlists](https://osu.ppy.sh/seasons/latest) * [competitivo](https://osu.ppy.sh/rankings/ranked-play/osu) * [desafio diário](https://osu.ppy.sh/rankings/daily-challenge) * [kudosu](https://osu.ppy.sh/rankings/kudosu) [comunidade] (https://osu.ppy.sh/community/forums) * [fórum](https://osu.ppy.sh/community/forums) * [chat](https://osu.ppy.sh/community/chat) * [concursos](https://osu.ppy.sh/community/contests) * [torneios](https://osu.ppy.sh/community/tournaments) * [transmissões ao vivo] (https://osu.ppy.sh/community/livestreams) * [desenvolvimento](https://discord.gg/ppy) [loja](https://osu.ppy.sh/store/listing) * [produtos] (https://osu.ppy.sh/store/listing) * [carrinho](https://osu.ppy.sh/store/cart) * [histórico de pedidos](https://osu.ppy.sh/store/orders) [ajuda] (https://osu.ppy.sh/wiki/pt-br/Main_page) * [wiki](https://osu.ppy.sh/wiki/pt-br/Main_page) * [perguntas frequentes](https://osu.ppy.sh/wiki/pt-br/FAQ) * [regras](https://osu.ppy.sh/wiki/pt-br/Rules) * [denunciar abuso](https://osu.ppy.sh/wiki/pt-br/Reporting_bad_behaviour/Abuse) (não, sério, preciso de ajuda!](https://osu.ppy.sh/wiki/pt-br/Help_centre) português brasileiro * [English](https://osu.ppy.sh/home/set-locale?locale=en) * [العربية] (https://osu.ppy.sh/home/set-locale?locale=ar) * [българска мова] (https://osu.ppy.sh/home/set-locale?locale=bg) * [български] (https://osu.ppy.sh/home/set-locale?locale=bg) * [català] (https://osu.ppy.sh/home/set-locale?locale=ca) * [česky] (https://osu.ppy.sh/home/set-locale?locale=cs) * [dansk] (https://osu.ppy.sh/home/set-locale?locale=da) * [Deutsch] (https://osu.ppy.sh/home/set-locale?locale=de) * [ελληνικά] (https://osu.ppy.sh/home/set-locale?locale=el) * [español] (https://osu.ppy.sh/home/set-locale?locale=es) * [español (Latinoamérica)] (https://osu.ppy.sh/home/set-locale?locale=es-419) * [suomi] (https://osu.ppy.sh/home/set-locale?locale=fi) * [wikang Filipino] (https://osu.ppy.sh/home/set-locale?locale=fil) * [français] (https://osu.ppy.sh/home/set-locale?locale=fr) * [עברית] (https://osu.ppy.sh/home/set-locale?locale=he) * [magyar] (https://osu.ppy.sh/home/set-locale?locale=hu) * [bahasa Indonesia] (https://osu.ppy.sh/home/set-locale?locale=id) * [italiano] (https://osu.ppy.sh/home/set-locale?locale=it) * [日本語] (https://osu.ppy.sh/home/set-locale?locale=ja) * [한국어] (https://osu.ppy.sh/home/set-locale?locale=ko) * [lietuvių kalba] (https://osu.ppy.sh/home/set-locale?locale=lt) * [latviešu valoda] (https://osu.ppy.sh/home/set-locale?locale=lv) * [bahasa Melayu] (https://osu.ppy.sh/home/set-locale?locale=ms) * [Nederlands] (https://osu.ppy.sh/home/set-locale?locale=nl) * [norsk] (https://osu.ppy.sh/home/set-locale?locale=no) * [polski] (https://osu.ppy.sh/home/set-locale?locale=pl) * [português] (https://osu.ppy.sh/home/set-locale?locale=pt) * [português brasileiro] (https://osu.ppy.sh/home/set-locale?locale=pt-br) * [română] (https://osu.ppy.sh/home/set-locale?locale=ro) * [pyckcku] (https://osu.ppy.sh/home/set-locale?locale=ru) * [slovenčina] (https://osu.ppy.sh/home/set-locale?locale=sk) * [slovenščina] (https://osu.ppy.sh/home/set-locale?locale=sl) * [српски] (https://osu.ppy.sh/home/set-locale?locale=sr) * [svenska] (https://osu.ppy.sh/home/set-locale?locale=sv) * [ไทย] (https://osu.ppy.sh/home/set-locale?locale=th) * [Türkçe] (https://osu.ppy.sh/home/set-locale?locale=tr) * [українська мова] (https://osu.ppy.sh/home/set-locale?locale=uk) * [tiếng Việt] (https://osu.ppy.sh/home/set-locale?locale=vi) * [简体中文] (https://osu.ppy.sh/home/set-locale?locale=zh) * [繁體中文 (台灣)] (https://osu.ppy.sh/home/set-locale?locale=zh-tw) [ir para o chat](https://osu.ppy.sh/community/chat) Sem notificações 1tudo1perfil [ver todas as notificações](https://osu.ppy.sh/notifications) Limpar todas [(https://osu.ppy.sh/users/21496977/osu#medals) Nova medalha "Cyclone" desbloqueado! [há 9 dias](https://osu.ppy.sh/users/21496977/osu#medals) [](https://osu.ppy.sh) [início](https://osu.ppy.sh) [notícias] (https://osu.ppy.sh/home/news) [equipe](https://osu.ppy.sh/wiki/pt-br/People/osu%21_team) [registro de alterações] (https://osu.ppy.sh/home/changelog) [baixar](https://osu.ppy.sh/home/download) [pesquisar](https://osu.ppy.sh/home/search) [beatmaps] (https://osu.ppy.sh/beatmapsets) [lista de beatmaps](https://osu.ppy.sh/beatmapsets) [artistas em destaque](https://osu.ppy.sh/beatmaps/artists) [pacotes de beatmap](https://osu.ppy.sh/beatmaps/packs) [colocações](https://osu.ppy.sh/rankings/osu/global) [global] (https://osu.ppy.sh/rankings/osu/global) [país](https://osu.ppy.sh/rankings/osu/country) [recordes](https://osu.ppy.sh/rankings/top-plays/osu) [equipe](https://osu.ppy.sh/rankings/osu/team) [playlists](https://osu.ppy.sh/seasons/latest) [competitivo](https://osu.ppy.sh/rankings/ranked-play/osu) [desafio diário](https://osu.ppy.sh/rankings/daily-challenge) [kudosu](https://osu.ppy.sh/rankings/kudosu) [comunidade] (https://osu.ppy.sh/community/forums) [fórum](https://osu.ppy.sh/community/forums) [chat](https://osu.ppy.sh/community/chat) [concursos] (https://osu.ppy.sh/community/contests) [torneios](https://osu.ppy.sh/community/tournaments) [transmissões ao vivo] (https://osu.ppy.sh/community/livestreams) [desenvolvimento](https://discord.gg/ppy) [loja](https://osu.ppy.sh/store/listing) [produtos] (https://osu.ppy.sh/store/listing) [carrinho](https://osu.ppy.sh/store/cart) [histórico de pedidos](https://osu.ppy.sh/store/orders) [ajuda] (https://osu.ppy.sh/wiki/pt-br/Main_page) [wiki](https://osu.ppy.sh/wiki/pt-br/Main_page) [perguntas frequentes](https://osu.ppy.sh/wiki/pt-br/FAQ) [regras] (https://osu.ppy.sh/wiki/pt-br/Rules) [denunciar abuso](https://osu.ppy.sh/wiki/pt-br/Reporting_bad_behaviour/Abuse) (não, sério, preciso de ajuda!](https://osu.ppy.sh/wiki/pt-br/Help_centre) [(https://osu.ppy.sh/home/search) [(https://osu.ppy.sh/wiki/Twitter) [(https://osu.ppy.sh/home/support "apoio o jogo") English العربية българска мова български català česky dansk Deutsch ελληνικά español español (Latinoamérica) suomi wikang Filipino français עברית magyar bahasa Indonesia italiano 日本語 한국어 lietuvių kalba latviešu valoda bahasa Melayu Nederlands norsk polski português português brasileiro română pyckcku slovenčina slovenščina српски svenska ไทย Türkçe українська мова tiếng Việt 简体中文 繁體中文 (台灣) [ir para o chat] (https://osu.ppy.sh/community/chat) Sem notificações [1](https://osu.ppy.sh/notifications) 1tudo1perfil [ver todas as notificações] (https://osu.ppy.sh/notifications) Limpar todas [(https://osu.ppy.sh/users/21496977/osu#medals) Nova medalha "Cyclone" desbloqueado! [há 9 dias](https://osu.ppy.sh/users/21496977/osu#medals) [(https://osu.ppy.sh/users/21496977) [!]/images/icons/profile.svg) Kr3Mega] (https://osu.ppy.sh/users/21496977) Modo Lazer [Meu Perfil](https://osu.ppy.sh/users/21496977) [Criar equipe](https://osu.ppy.sh/teams/create) [Amigos](https://osu.ppy.sh/home/friends) [Listas de observação](https://osu.ppy.sh/home/follows/forum_topic) [Configurações] (https://osu.ppy.sh/home/account/edit) Desconectar partidas #### 6WC26'ML: (Brazil B) vs (United States B) mostrar mais 23:12:53 [he barks for me](https://osu.ppy.sh/users/19499604) saiu da partida 23:13:07 [TheMatrix313](https://osu.ppy.sh/users/24500538) saiu da partida 23:13:17 [Sayo C8H18](https://osu.ppy.sh/users/19951674) entrou na partida 23:13:18 [he barks for me](https://osu.ppy.sh/users/19499604) entrou na partida 23:13:23 [regisnalin](https://osu.ppy.sh/users/15378480) saiu da partida 23:13:56 [lipeeexper](https://osu.ppy.sh/users/12613518) entrou na partida 23:15:25 [jobbiested](https://osu.ppy.sh/users/20811344) saiu da partida 23:15:28 [jobbiested](https://osu.ppy.sh/users/20811344) entrou na partida [23:15:47 - 23:18:13osu!Pontuação V2 # NIGHTMARE INVITATION [feringira's Insane!] ## EBIMAYO] (https://osu.ppy.sh/beatmaps/5337775) [(https://osu.ppy.sh/rankings/osu/performance?country=US)[he barks for me] (https://osu.ppy.sh/users/19499604) Combo953 Precisão96,64% Pontuação975.996 great547 ok29 meh0 Erros0 [(https://osu.ppy.sh/rankings/osu/performance?country=US)[ityoluckyday](https://osu.ppy.sh/users/11037790) Combo793 Precisão93,95% Pontuação686.863 great524 ok51 meh1 Erros0 [(https://osu.ppy.sh/rankings/osu/performance?country=US)[jobbiested]

(<https://osu.ppy.sh/users/20811344>) Combo747 Precisão95,28% Pontuação678.423 great536 ok38 meh1 Erros1 [] (<https://osu.ppy.sh/rankings/osu/performance?country=US>)[Dire_Spidar](<https://osu.ppy.sh/users/17927975>) Combo505 Precisão95,78% Pontuação567.533 great541 ok29 meh6 Erros0 [](<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[AlemaoG] (<https://osu.ppy.sh/users/16163696>) Combo794 Precisão94,10% Pontuação703.434 great526 ok48 meh0 Erros2 [] (<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[LegitsNeverBan](<https://osu.ppy.sh/users/17203125>) Combo561 Precisão91,90% Pontuação448.784 great507 ok66 meh2 Erros1 [](<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[Sayo C8H18] (<https://osu.ppy.sh/users/19951674>) Combo467 Precisão93,32% Pontuação401.261 great521 ok47 meh5 Erros3 [] (<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[lipeexper](<https://osu.ppy.sh/users/12613518>) Combo455 Precisão89,29% Pontuação345.628 great486 ok84 meh2 Erros4 Equipe Vermelha1.899.107 Equipe Azul2.908.815 **Equipe Azul venceu** por 1.009.708 23:18:20 [regisnalin](<https://osu.ppy.sh/users/15378480>) entrou na partida 23:18:29 [lipeexper](<https://osu.ppy.sh/users/12613518>) saiu da partida 23:18:32 [CraneTrek](<https://osu.ppy.sh/users/6389632>) entrou na partida 23:19:42 [CraneTrek](<https://osu.ppy.sh/users/6389632>) saiu da partida 23:19:42 [obbisted](<https://osu.ppy.sh/users/20811344>) saiu da partida 23:20:29 [ShinySpeal](<https://osu.ppy.sh/users/33306913>) entrou na partida 23:20:37 [TheMatrix313](<https://osu.ppy.sh/users/24500538>) entrou na partida 23:20:50 [he barks for me] (<https://osu.ppy.sh/users/19499604>) saiu da partida [23:21:09 - 23:24:53osu!Pontuação V2 # Juubaku \[Nathan's Extra\] ## kalon.] (<https://osu.ppy.sh/beatmaps/1123160>) [](<https://osu.ppy.sh/rankings/osu/performance?country=US>)[ityoluckyday] (<https://osu.ppy.sh/users/11037790>) Combo1.123 Precisão99,22% Pontuação984.176 great762 ok9 meh0 Erros0 [] (<https://osu.ppy.sh/rankings/osu/performance?country=US>)[TheMatrix313](<https://osu.ppy.sh/users/24500538>) Combo953 Precisão97,77% Pontuação748.457 great746 ok23 meh1 Erros1 [](<https://osu.ppy.sh/rankings/osu/performance?country=US>)[Dire_Spidar] (<https://osu.ppy.sh/users/17927975>) Combo758 Precisão97,38% Pontuação607.105 great745 ok12 meh11 Erros3 [] (<https://osu.ppy.sh/rankings/osu/performance?country=US>)[ShinySpeal](<https://osu.ppy.sh/users/33306913>) Combo624 Precisão98,55% Pontuação537.428 great756 ok11 meh1 Erros3 [](<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[AlemaoG] (<https://osu.ppy.sh/users/16163696>) Combo734 Precisão98,77% Pontuação617.843 great758 ok10 meh1 Erros2 [] (<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[Sayo C8H18](<https://osu.ppy.sh/users/19951674>) Combo578 Precisão98,85% Pontuação527.629 great760 ok5 meh3 Erros3 [](<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[LegitsNeverBan] (<https://osu.ppy.sh/users/17203125>) Combo578 Precisão97,38% Pontuação463.995 great745 ok15 meh5 Erros6 [] (<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[regisnalin](<https://osu.ppy.sh/users/15378480>) Combo436 Precisão97,82% Pontuação426.610 great748 ok18 meh1 Erros4 Equipe Vermelha2.036.077 Equipe Azul2.877.166 **Equipe Azul venceu** por 841.089 23:24:48 [he barks for me](<https://osu.ppy.sh/users/19499604>) entrou na partida 23:25:09 [obbisted](<https://osu.ppy.sh/users/20811344>) entrou na partida 23:25:11 [ShinySpeal](<https://osu.ppy.sh/users/33306913>) saiu da partida 23:25:12 [TheMatrix313] (<https://osu.ppy.sh/users/24500538>) saiu da partida 23:25:29 [regisnalin](<https://osu.ppy.sh/users/15378480>) saiu da partida 23:25:43 [he barks for me](<https://osu.ppy.sh/users/19499604>) saiu da partida 23:25:56 [he barks for me](<https://osu.ppy.sh/users/19499604>) entrou na partida 23:26:31 [AugustoDroid](<https://osu.ppy.sh/users/17037404>) entrou na partida [23:26:54 - 23:29:33osu!Pontuação V2 # Stardust \[Crusade\] ## Duskfox](<https://osu.ppy.sh/beatmaps/5869470>) [](<https://osu.ppy.sh/rankings/osu/performance?country=US>)[Dire_Spidar] (<https://osu.ppy.sh/users/17927975>) Combo671 Precisão96,10% Pontuação670.761 great420 ok22 meh2 Erros1 [] (<https://osu.ppy.sh/rankings/osu/performance?country=US>)[ityoluckyday](<https://osu.ppy.sh/users/11037790>) Combo684 Precisão96,29% Pontuação666.349 great421 ok21 meh3 Erros0 [](<https://osu.ppy.sh/rankings/osu/performance?country=US>)[he barks for me] (<https://osu.ppy.sh/users/19499604>) Combo485 Precisão98,35% Pontuação593.298 great435 ok8 meh0 Erros2 [] (<https://osu.ppy.sh/rankings/osu/performance?country=US>)[obbisted](<https://osu.ppy.sh/users/20811344>) Combo460 Precisão95,84% Pontuação486.297 great418 ok24 meh3 Erros0 [](<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[Sayo C8H18] (<https://osu.ppy.sh/users/19951674>) Combo716 Precisão95,62% Pontuação701.836 great416 ok28 meh1 Erros0 [] (<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[AlemaoG](<https://osu.ppy.sh/users/16163696>) Combo408 Precisão94,12% Pontuação460.062 great407 ok34 meh3 Erros1 [](<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[LegitsNeverBan] (<https://osu.ppy.sh/users/17203125>) Combo226 Precisão91,35% Pontuação271.317 great390 ok48 meh3 Erros4 [] (<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[AugustoDroid](<https://osu.ppy.sh/users/17037404>) Combo145 Precisão90,71% Pontuação217.787 great387 ok44 meh12 Erros2 Equipe Vermelha1.651.002 Equipe Azul2.416.705 **Equipe Azul venceu** por 765.703 23:29:37 [CraneTrek](<https://osu.ppy.sh/users/6389632>) entrou na partida 23:29:40 [regisnalin](<https://osu.ppy.sh/users/15378480>) entrou na partida 23:29:43 [ShinySpeal](<https://osu.ppy.sh/users/33306913>) entrou na partida 23:30:14 [Sayo C8H18] (<https://osu.ppy.sh/users/19951674>) saiu da partida 23:30:41 [ShinySpeal](<https://osu.ppy.sh/users/33306913>) saiu da partida 23:30:42 [obbisted](<https://osu.ppy.sh/users/20811344>) saiu da partida 23:30:46 [ityoluckyday](<https://osu.ppy.sh/users/11037790>) saiu da partida 23:30:53 [Dire_Spidar](<https://osu.ppy.sh/users/17927975>) saiu da partida 23:31:41 [TheMatrix313](<https://osu.ppy.sh/users/24500538>) entrou na partida 23:31:52 [ityoluckyday](<https://osu.ppy.sh/users/11037790>) entrou na partida 23:32:05 [stinkypanda_] (<https://osu.ppy.sh/users/25445714>) entrou na partida 23:32:11 [he barks for me](<https://osu.ppy.sh/users/19499604>) saiu da partida [23:32:29 - 23:34:45osu!Pontuação V2 # <header> \[<insane diffMapper>=Phen02>\] ## Shione Kitsune](<https://osu.ppy.sh/beatmaps/5476229>) [] (<https://osu.ppy.sh/rankings/osu/performance?country=US>)[CraneTrek](<https://osu.ppy.sh/users/6389632>) Combo550 Precisão93,01% Pontuação491.346 great434 ok30 meh9 Erros6 [](<https://osu.ppy.sh/rankings/osu/performance?country=US>)[TheMatrix313] (<https://osu.ppy.sh/users/24500538>) Combo553 Precisão90,33% Pontuação468.133 great411 ok62 meh6 Erros0 [] (<https://osu.ppy.sh/rankings/osu/performance?country=US>)[ityoluckyday](<https://osu.ppy.sh/users/11037790>) Combo452 Precisão94,19% Pontuação458.779 great440 ok30 meh7 Erros2 [](<https://osu.ppy.sh/rankings/osu/performance?country=US>)[stinkypanda_] (<https://osu.ppy.sh/users/25445714>) Combo449 Precisão86,43% Pontuação313.692 great391 ok60 meh18 Erros10 [] (<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[AugustoDroid](<https://osu.ppy.sh/users/17037404>) Combo746 Precisão85,42% Pontuação623.305 great376 ok97 meh5 Erros1 [](<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[LegitsNeverBan] (<https://osu.ppy.sh/users/17203125>) Combo519 Precisão88,07% Pontuação427.069 great399 ok59 meh19 Erros2 [] (<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[AlemaoG](<https://osu.ppy.sh/users/16163696>) Combo406 Precisão88,34% Pontuação334.427 great402 ok52 meh23 Erros2 [](<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[regisnalin] (<https://osu.ppy.sh/users/15378480>) Combo284 Precisão87,30% Pontuação261.093 great390 ok81 meh7 Erros1 Equipe Vermelha1.645.894 Equipe Azul1.731.950 **Equipe Azul venceu** por 86.056 23:34:55 [he barks for me](<https://osu.ppy.sh/users/19499604>) entrou na partida 23:35:10 [CraneTrek](<https://osu.ppy.sh/users/6389632>) saiu da partida 23:35:28 [ityoluckyday](<https://osu.ppy.sh/users/11037790>) saiu da partida 23:35:30 [stinkypanda_](<https://osu.ppy.sh/users/25445714>) saiu da partida 23:35:40 [TheMatrix313] (<https://osu.ppy.sh/users/24500538>) saiu da partida 23:36:08 [ityoluckyday](<https://osu.ppy.sh/users/11037790>) entrou na partida 23:36:10 [Dire_Spidar](<https://osu.ppy.sh/users/17927975>) entrou na partida 23:36:22 [AugustoDroid](<https://osu.ppy.sh/users/17037404>) saiu da partida 23:36:25 [regisnalin](<https://osu.ppy.sh/users/15378480>) saiu da partida 23:36:29 [Sayo C8H18](<https://osu.ppy.sh/users/19951674>) entrou na partida 23:36:56 [lipeexper](<https://osu.ppy.sh/users/12613518>) entrou na partida 23:36:59 [ShinySpeal](<https://osu.ppy.sh/users/33306913>) entrou na partida 23:37:07 [ityoluckyday](<https://osu.ppy.sh/users/11037790>) saiu da partida 23:37:12 [ityoluckyday] (<https://osu.ppy.sh/users/11037790>) entrou na partida [23:37:33 - 23:40:15osu!Pontuação V2 # True Friends \[Insane\] ## Bring Me the Horizon](<https://osu.ppy.sh/beatmaps/5869413>) [](<https://osu.ppy.sh/rankings/osu/performance?country=US>)[he barks for me] (<https://osu.ppy.sh/users/19499604>) Combo986 Precisão97,92% Pontuação1.113.267 great621 ok20 meh0 Erros0 [] (<https://osu.ppy.sh/rankings/osu/performance?country=US>)[ityoluckyday](<https://osu.ppy.sh/users/11037790>) Combo872 Precisão97,06% Pontuação911.374 great613 ok27 meh1 Erros0 [](<https://osu.ppy.sh/rankings/osu/performance?country=US>)[Dire_Spidar] (<https://osu.ppy.sh/users/17927975>) Combo824 Precisão97,69% Pontuação880.818 great619 ok21 meh1 Erros0 [] (<https://osu.ppy.sh/rankings/osu/performance?country=US>)[ShinySpeal](<https://osu.ppy.sh/users/33306913>) Combo762 Precisão98,13% Pontuação822.616 great624 ok15 meh0 Erros2 [](<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[AlemaoG] (<https://osu.ppy.sh/users/16163696>) Combo728 Precisão95,35% Pontuação691.218 great598 ok39 meh1 Erros3 [] (<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[LegitsNeverBan](<https://osu.ppy.sh/users/17203125>) Combo202 Precisão93,99% Pontuação344.045 great590 ok34 meh7 Erros10 [](<https://osu.ppy.sh/rankings/osu/performance?country=BR>)[lipeexper]

(https://osu.ppy.sh/users/12613518) Combo173 Precisão95,76% Pontuação335.733 great607 ok16 meh9 Erros9 []
 (https://osu.ppy.sh/rankings/osu/performance?country=BR)[Sayo C8H18](https://osu.ppy.sh/users/19951674) Combo337 Precisão87,08% Pontuação255.926 great524 ok97 meh11 Erros9 Equipe Vermelha1.626.922 Equipe Azul3.728.075 **Equipe Azul venceu** por 2.101.153
 23:40:25 [regisnalin](https://osu.ppy.sh/users/15378480) entrou na partida 23:40:40 [Jazzinhaa](https://osu.ppy.sh/users/19174506) entrou na partida 23:40:49 [ityoluckyday](https://osu.ppy.sh/users/11037790) saiu da partida 23:40:52 [obbiedsted](https://osu.ppy.sh/users/20811344) entrou na partida 23:41:03 [AugustoDroid](https://osu.ppy.sh/users/17037404) entrou na partida 23:41:12 [TheMatrix313] (https://osu.ppy.sh/users/24500538) entrou na partida 23:41:13 [Jazzinhaa](https://osu.ppy.sh/users/19174506) saiu da partida 23:41:15 [lipeexper](https://osu.ppy.sh/users/12613518) saiu da partida 23:41:16 [ityoluckyday](https://osu.ppy.sh/users/11037790) entrou na partida 23:41:42 [obbiedsted](https://osu.ppy.sh/users/20811344) saiu da partida 23:42:06 [ityoluckyday](https://osu.ppy.sh/users/11037790) saiu da partida 23:42:14 [AugustoDroid](https://osu.ppy.sh/users/17037404) saiu da partida [23:42:43 - 23:46:38osu!Pontuação V2 # Buta Musou \ [Kaguya_ Sama's Insane] \ ## katagiri](https://osu.ppy.sh/beatmaps/3653688) [] (https://osu.ppy.sh/rankings/osu/performance?country=US) [TheMatrix313](https://osu.ppy.sh/users/24500538) Combo1.349 Precisão98,85% Pontuação1.019.718 great797 ok14 meh0 Erros0 []
 (https://osu.ppy.sh/rankings/osu/performance?country=US)[he barks for me](https://osu.ppy.sh/users/19499604) Combo1.348 Precisão97,45% Pontuação1.009.220 great780 ok31 meh0 Erros0 [] (https://osu.ppy.sh/rankings/osu/performance?country=US)[Dire_ Spidar] (https://osu.ppy.sh/users/17927975) Combo1.348 Precisão92,91% Pontuação927.667 great726 ok80 meh5 Erros0 []
 (https://osu.ppy.sh/rankings/osu/performance?country=US)[ShinySpeal](https://osu.ppy.sh/users/33306913) Combo1.220 Precisão99,30% Pontuação855.215 great803 ok6 meh2 Erros0 [] (https://osu.ppy.sh/rankings/osu/performance?country=BR)[LegitsNeverBan] (https://osu.ppy.sh/users/17203125) Combo1.348 Precisão99,26% Pontuação1.034.368 great802 ok9 meh0 Erros0 []
 (https://osu.ppy.sh/rankings/osu/performance?country=BR)[AlemaoG](https://osu.ppy.sh/users/16163696) Combo1.346 Precisão98,60% Pontuação952.248 great794 ok17 meh0 Erros0 [] (https://osu.ppy.sh/rankings/osu/performance?country=BR)[regisnalin] (https://osu.ppy.sh/users/15378480) Combo440 Precisão98,03% Pontuação487.145 great788 ok19 meh4 Erros0 []
 (https://osu.ppy.sh/rankings/osu/performance?country=BR)[Sayo C8H18](https://osu.ppy.sh/users/19951674) Combo664 Precisão92,27% Pontuação382.912 great720 ok83 meh4 Erros4 Equipe Vermelha2.856.673 Equipe Azul3.811.820 **Equipe Azul venceu** por 955.147 23:46:47 [ityoluckyday](https://osu.ppy.sh/users/11037790) entrou na partida 23:47:12 [obbiedsted](https://osu.ppy.sh/users/20811344) entrou na partida 23:47:22 [Sayo C8H18](https://osu.ppy.sh/users/19951674) saiu da partida 23:47:27 [ShinySpeal](https://osu.ppy.sh/users/33306913) saiu da partida 23:47:34 [AugustoDroid](https://osu.ppy.sh/users/17037404) entrou na partida 23:47:59 [TheMatrix313] (https://osu.ppy.sh/users/24500538) saiu da partida 23:48:24 [lipeexper](https://osu.ppy.sh/users/12613518) entrou na partida 23:48:38 [regisnalin](https://osu.ppy.sh/users/15378480) saiu da partida 23:50:06 [AugustoDroid](https://osu.ppy.sh/users/17037404) saiu da partida 23:50:13 [AugustoDroid](https://osu.ppy.sh/users/17037404) entrou na partida [23:51:22 - 23:55:03osu!Pontuação V2 # Zero Talking \ [Insane] \ ## Harumaki Gohan](https://osu.ppy.sh/beatmaps/4805944) [] (https://osu.ppy.sh/rankings/osu/performance?country=US)[obbiedsted] (https://osu.ppy.sh/users/20811344) Combo1.343 Precisão93,27% Pontuação886.632 great739 ok83 meh0 Erros0 []
 (https://osu.ppy.sh/rankings/osu/performance?country=US)[he barks for me](https://osu.ppy.sh/users/19499604) Combo774 Precisão93,35% Pontuação531.075 great741 ok78 meh2 Erros1 [] (https://osu.ppy.sh/rankings/osu/performance?country=US)[Dire_ Spidar] (https://osu.ppy.sh/users/17927975) Combo751 Precisão91,55% Pontuação509.990 great718 ok103 meh1 Erros0 []
 (https://osu.ppy.sh/rankings/osu/performance?country=US)[ityoluckyday](https://osu.ppy.sh/users/11037790) Combo733 Precisão85,62% Pontuação381.253 great655 ok143 meh7 Erros17 [] (https://osu.ppy.sh/rankings/osu/performance?country=BR)[AlemaoG] (https://osu.ppy.sh/users/16163696) Combo1.345 Precisão79,99% Pontuação669.721 great576 ok243 meh3 Erros0 []
 (https://osu.ppy.sh/rankings/osu/performance?country=BR)[LegitsNeverBan](https://osu.ppy.sh/users/17203125) Combo1.030 Precisão81,85% Pontuação462.114 great599 ok220 meh3 Erros0 [] (https://osu.ppy.sh/rankings/osu/performance?country=BR)[lipeexper] (https://osu.ppy.sh/users/12613518) Combo910 Precisão84,87% Pontuação447.361 great636 ok184 meh2 Erros0 []
 (https://osu.ppy.sh/rankings/osu/performance?country=BR)[AugustoDroid](https://osu.ppy.sh/users/17037404) Combo399 Precisão85,93% Pontuação219.225 great652 ok160 meh6 Erros4 Equipe Vermelha1.798.421 Equipe Azul2.308.950 **Equipe Azul venceu** por 510.529 23:55:25 [regisnalin](https://osu.ppy.sh/users/15378480) entrou na partida 23:55:27 [ShinySpeal](https://osu.ppy.sh/users/33306913) entrou na partida 23:55:33 [TheMatrix313](https://osu.ppy.sh/users/24500538) entrou na partida 23:55:49 [Sayo C8H18] (https://osu.ppy.sh/users/19951674) entrou na partida 23:55:54 [ShinySpeal](https://osu.ppy.sh/users/33306913) saiu da partida 23:56:00 [ityoluckyday](https://osu.ppy.sh/users/11037790) saiu da partida 23:56:23 [obbiedsted](https://osu.ppy.sh/users/20811344) saiu da partida 23:56:42 [AlemaoG](https://osu.ppy.sh/users/16163696) saiu da partida 23:56:43 [regisnalin](https://osu.ppy.sh/users/15378480) saiu da partida 23:56:52 [TheMatrix313](https://osu.ppy.sh/users/24500538) saiu da partida 23:56:57 [lipeexper](https://osu.ppy.sh/users/12613518) saiu da partida 23:58:18 [he barks for me](https://osu.ppy.sh/users/19499604) saiu da partida [00:00:30 - 00:06:05osu!Pontuação V2 # sympathy \ [never let me go >||<] \ ## OsushiFreak & ToffU](https://osu.ppy.sh/beatmaps/5869454) [] (https://osu.ppy.sh/rankings/osu/performance?country=US)[Dire_ Spidar] (https://osu.ppy.sh/users/17927975) Combo1.281 Precisão97,59% Pontuação522.450 great1.464 ok43 meh7 Erros2 [] (https://osu.ppy.sh/rankings/osu/performance?country=BR)[Sayo C8H18](https://osu.ppy.sh/users/19951674) Combo818 Precisão97,54% Pontuação379.663 great1.464 ok41 meh6 Erros5 [] (https://osu.ppy.sh/rankings/osu/performance?country=BR)[LegitsNeverBan] (https://osu.ppy.sh/users/17203125) Combo613 Precisão96,70% Pontuação364.839 great1.448 ok48 meh12 Erros8 []
 (https://osu.ppy.sh/rankings/osu/performance?country=BR)[AugustoDroid](https://osu.ppy.sh/users/17037404) Combo958 Precisão93,74% Pontuação347.937 great1.384 ok107 meh9 Erros16 Equipe Vermelha712.776 Equipe Azul902.113 **Equipe Azul venceu** por 189.337 00:06:32 [LegitsNeverBan](https://osu.ppy.sh/users/17203125) saiu da partida 00:06:41 [Dire_ Spidar](https://osu.ppy.sh/users/17927975) saiu da partida 00:06:52 [AugustoDroid](https://osu.ppy.sh/users/17037404) saiu da partida 00:06:54 [Dire_ Spidar] (https://osu.ppy.sh/users/17927975) entrou na partida 00:07:08 [Sayo C8H18](https://osu.ppy.sh/users/19951674) saiu da partida 00:07:21 a partida foi desfeita / > > >

Response: O maior erro de quem desenvolve ferramentas para torneios de osu! é tentar criar uma plataforma "tudo-em-um" (inscrição, chaveamento, mappool e referee) para competir com o Google Sheets. A comunidade é viciada em planilhas porque nenhuma interface web oferece a mesma flexibilidade em tempo de execução quando imprevistos acontecem.

Para ter tração real e relevância técnica, foque em **gargalos que o Google Sheets e a infraestrutura padrão não conseguem resolver eficientemente**.

Abaixo estão quatro ideias de alto impacto técnico, arquiteturalmente viáveis via API v2 e Bancho:

1. Match Link Parser & Hub de Inteligência Competitiva (Scouting / Scrims)

O log de partida enviado (6WC: Brasil B vs EUA B) expõe um problema diário de capitães e analistas: interpretar a rotação de jogadores, inconsistência sob pressão e desempenho por slot de mod.

- **O que faz:** Ingestão de IDs/URLs de partidas (/community/matches/:id). A ferramenta cruza os dados brutos da partida com os dados do beatmap via API v2 e constrói o relatório analítico do confronto.
- **Métricas que ninguém calcula automaticamente:**
 - **Consistency Drop:** Diferença percentual de accuracy e misses de um jogador entre solo play/qualifiers e match multiplayer sob ScoreV2.

- **Team Delta per Mod:** Identificação de dependência de "carregador" (ex: jogador X gerando 40% da pontuação total do time em mapas de alta densidade de leitura vs mapas de velocidade).
- **Warmup & Pick Detection:** Heurística automática que descarta warmups e classifica a ordem de picks/bans baseada no mappool cadastrado.
- **Stack/Arquitetura:** Ingestão assíncrona com fila (BullMQ/Redis) para contornar o rate limit da API v2 da ppy; PostgreSQL com modelo relacional normalizado para armazenar partidas históricas e cruzar confrontos diretos entre países/times.

2. Mappool Balancer & Playtesting Analytics Platform

Mappoolers e playtesters dependem de formulários manuais do Google Forms e comentários soltos no Discord. O balanceamento de pools muitas vezes erra a mão na curva de dificuldade entre estágios (RO32, RO16, Quarterfinals, etc.).

- **O que faz:** Plataforma dedicada para pools fechadas onde playtesters jogam os mapas e a ferramenta coleta os replays ou monitora os lobbies de teste.
- **Diferenciais analíticos:**
 - **Difficulty Spike Analysis:** Mapeamento de density graph (strains de aim, tap, reading) do beatmap contra a distribuição de misses dos testers.
 - **Stat Radar:** Comparação dos mapas selecionados para o pool contra a média histórica da faixa de rank do torneio (ex: 6-digit, 5-digit, BWS).
 - **Predictor de Win-Rate por Mod:** Algoritmo que calcula, com base nos perfis dos inscritos, quais mods têm maior probabilidade de serem "perma-banidos" por desbalanceamento estrutural.

3. Engine de Overlays Dinâmicos com WebSockets (Casting / Streaming)

Ferramentas locais de overlay (como StreamCompanion ou tosu) funcionam bem para o streamer solo, mas em torneios com produção remota (caster em um país, streamer em outro, referee em um terceiro), a sincronização é caótica.

- **O que faz:** Um servidor intermediário que consome eventos do Bancho IRC (ou polling contínuo da partida via API v2) e expõe um WebSocket centralizado para overlays de OBS.
- **Recursos:**
 - Cenas de Pick/Ban atualizadas em tempo real sem depender de um operador no Sheets.
 - Score differential preditivo em tempo real com micro-animações no browser source do OBS.
 - Roster Cards gerados sob demanda puxando estatísticas de performance acumuladas durante o torneio para os jogadores atualmente no lobby.

4. Motor Automatizado de Screening & Rank Drift (Anti-Smurf / BWS)

O processo de triagem (*screening*) de torneios de faixas de rank (como torneios 6-digit ou 5-digit) é amplamente manual e sujeito a erros de prazo.

- **O que faz:** Um pipeline que automatiza verificações de elegibilidade contínuas desde a inscrição até o fim da fase de registro.
- **Cálculos e Heurísticas:**
 - **BWS Dinâmico:** Aplicação da fórmula de Badge-Weighted Seeding:
$$BWS = \text{rank}^{0.9937^{\text{badges}^2}}$$
com parsing automático de badges no perfil para filtrar badges não-competitivas (badges de mapping, modding ou contribuição).
 - **Deranking Heuristics:** Detecção de contas suspeitas (jogadores que limpam top plays, playtime alto recente sem ganho de rank equivalente, ou spikes anômalos de accuracy em streams/jumps em torneios anteriores).

Por onde começar (Recomendação Estratégica)

Não comece construindo gestão de chaves ou cadastros complexos. O produto com **menor atrito de adoção e maior valor imediato** é o **Match Link Parser & Scrim Analyzer (Ideia 1)**.

1. **Entrada simples:** O usuário só precisa colar uma lista de links de partidas do Bancho (como a do Brasil B vs US B).
2. **Saída de alto valor:** Gera um dashboard comparativo da lineup, aponta fragilidades nos picks de FreeMod/Hidden e entrega valor sem exigir que o organizador abandone as planilhas existentes dele.
3. **Evolução:** Uma vez que as equipes usam seu parser para treinar, a extensão natural é integrar o agendamento de scrims e, posteriormente, a gestão oficial do torneio.

User prompt: E se eu criar um banco de dados com essas informações extraídas diretamente desses mplinks e meu sistema online criado em react cria skills (Tipo videogame) com gráficos de barra bonitos apresentando os melhores mods de cada time, pontos fortes e fracos? Um sistema de auditoria completo com todos os mplinks do torneio e uma trajetória linear pós torneio contando o que aconteceu e os comportamentos de cada time de forma ledária ou dramática?

Response: A ideia é conceitualmente forte para retenção de usuários, mas possui duas armadilhas críticas de engenharia e produto que precisam ser desarmadas antes de você escrever a primeira linha de código:

- Atributos ingênuos geram estatísticas inúteis:** Atribuir barras de *skill* baseadas em média simples de Accuracy ou Pontuação ScoreV2 é matematicamente falho no *osu!*. Se o Time A enfrentou um pool focado em *gimmick/low AR* e o Time B enfrentou um pool focado em *flow aim*, as notas brutas não são comparáveis.
- Storytelling genérico vira "fluff" de IA:** Se a narrativa for um gerador de texto com tom épico genérico ("*O Brasil lutou bravamente nas sombras...*"), o público competitivo vai ignorar. O valor real reside em uma **crônica esportiva analítica orientada a eventos** (estilo relatórios da HLTV no CS ou recaps analíticos da NBA).

Abaixo está o detalhamento de como estruturar essa solução com rigor técnico e arquitetural.

1. Engenharia do Sistema de "Skills" (Modelagem Matemática)

Para transformar dados de ScoreV2 em atributos reais de RPG (ex: *Aim*, *Speed*, *Finger Control*, *Reading*, *Clutch*), você não pode depender apenas do payload básico do endpoint de multiplayer (`/matches/:id`).

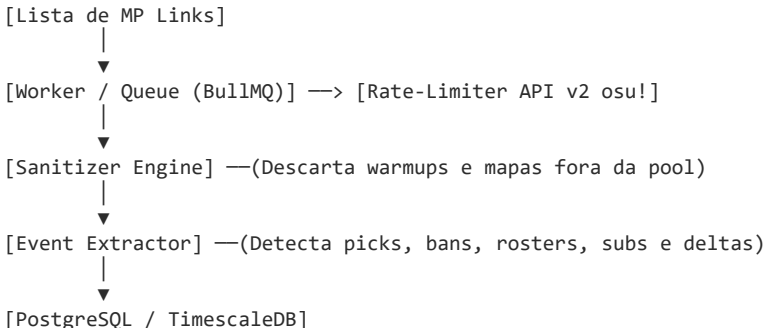
- Decomposição do Beatmap:** O endpoint do Bancho dá apenas o `beatmap_id` e o score. Seu backend precisa rodar uma biblioteca de cálculo de dificuldade (como `rosu-pp` via Rust/Node addon ou processar as strains de Aim vs Speed da API v2) para classificar o mapa em vetores de habilidades.
- Normalização por Z-Score ou Desvio do Pool:** Em vez de pontuação absoluta, a habilidade de um time em um mod (ex: HardRock) deve ser medida pelo desvio padrão em relação a todos os outros times que jogaram aquele mesmo mapa no torneio:

$$Z = \frac{Score_{time} - \mu_{mapa}}{\sigma_{mapa}}$$

- Definição dos Atributos de RPG:**
 - Precision / Aim:** Performance em slots HR e NM1.
 - Agility / Speed:** Performance em DT e mapas com strain de tapping elevado.
 - Consistency:** Relação entre Combo Máximo obtido vs Combo Máximo possível, ponderado pelo número de missbreaks.
 - Tenacity / Clutch Factor:** Desempenho relativo em mapas decisivos (Match Point ou Tiebreakers) comparado à média normal do time.

2. Pipeline de Auditoria e Ingestão de Dados

Torneios reais têm partidas caóticas: jogadores entram e saem do lobby, dão warmups de 7 minutos que não contam, desconectam no meio do mapa ou pedem abort por lag.



- Sanitização Obrigatória:**
 - Filtro de Warmup:** Cruza o `beatmap_id` da partida com a tabela de beatmaps oficiais cadastrados no torneio. Se não bater, é descartado do cálculo de skills.
 - Deteção de Roll:** Identifica as mensagens de texto do chat do lobby (caso disponíveis pelo Bancho) ou o primeiro mapa selecionado para traçar a ordem de *picks*.
 - Substituições:** Mapear a rotação de jogadores por round (ex: jogador X só entrou no slot de Hidden e FreeMod).

3. Storytelling Baseado em Eventos (Narrativa Orientada a Dados)

Para que a "trajetória linear pós-torneio" seja legitimamente impactante, você não deve alimentar uma LLM com dados crus. Em vez disso, construa uma camada determinística que detecta **marcos dramáticos objetivos** antes de gerar qualquer texto:

- Deteção de "Choke":** Um time liderava por mais de 150.000 pontos até os 80% do mapa e perdeu por erro em cadeia no final slider/stream.
- Deteção de "Hard Carry":** Um único jogador foi responsável por mais de 35% do score agregado de uma lineup 4v4 em determinado mod.
- Deteção de "Mappool Counter-Pick":** O Time A escolheu o mod historicamente mais fraco do Time B e venceu com margem recorde.
- A Conexão com LLMs:** Passe esses eventos estruturados em JSON para o modelo de linguagem:

```
{
  "stage": "Semifinals",
  "match": "Brazil B vs US B",
  "turning_point": "Map 4 (<header>)",
  "event_type": "Clutch",
  "delta_score": 86056,
  "key_player": "AugustoDroid",
  "historical_context": "US B era favorito com 88% de winrate em Tech"
}
```

Isso garante que a crônica seja factual, focada na tensão competitiva e livre de alucinações vazias.

4. Arquitetura Frontend (React)

Para visualização no frontend, gráficos de barra simples são insuficientes para atributos multidimensionais.

- **Radar Charts (Stat Hexagons):** Utilize gráficos de radar para os atributos do time (estilo *Pokemon* ou *FIFA Cards*). Permite sobrepor diretamente o hexágono do Time A contra o Time B para identificar sobreposições e disparidades óbvias antes do confronto.
- **Linha do Tempo Interativa (Match Timeline):**
 - Um gráfico contínuo de momentum da partida (gráfico de área com score diferencial rodada a rodada).
 - Marcadores nos nós do gráfico indicando quem deu o pick, quem sofreu o "break" e onde ocorreram os sweeps de mapas seguidos.
- **Card de Roster Dinâmico:** Ao clicar em um jogador dentro do time, o radar chart atualiza para refletir apenas o impacto individual dele na rotação da equipe.

Próximo Passo Prático

Antes de construir o frontend ou integrar LLMs:

1. Monte um script simples em Node.js ou Python que receba o link da partida acima ([<https://osu.ppy.sh/community/matches/121918084>]([<https://osu.ppy.sh/community/matches/121918084>])).
2. Consuma o JSON da API v2 e escreva o algoritmo para extrair apenas:
 - Score total por equipe em cada round.
 - Jogador com maior acurácia e jogador com maior combo em cada time.
 - Diferença percentual entre as equipes por tipo de mapa.

Validar a qualidade da extração bruta é o gargalo que vai ditar se as visualizações posteriores serão confiáveis ou puro ruído visual.

User prompt: Na verdade eu acho que talvez seria melhor ainda eu criar um app que pega beatmaps e detecta quais skills ele tem. Isso é possível?

Response: Sim, é totalmente possível. Tecnicamente, inclusive, essa é uma decisão de engenharia muito superior: em vez de tentar adivinhar a habilidade de um time olhando para números agregados de partidas, você resolve o problema na raiz caracterizando matematicamente o estímulo (o beatmap).

Para construir isso, você não precisa de IA generativa. O mapa no *osu!* é um arquivo de texto estruturado (.osu) contendo coordenadas cartesianas (x, y), marcações temporais em milissegundos (t) e vetores de interpolação de sliders. Toda skill pode ser extraída via análise vetorial e temporal desses dados.

1. Como transformar dados do .osu em Skills Matemáticas

A engine oficial do *osu!* (*osu-tools*) e bibliotecas de performance (como *rosu-pp*) já fazem parte desse cálculo através de **Strain Peaks** (picos de tensão). Para extrair skills específicas, você decompõe as sequências de objetos ($P_n = [x_n, y_n, t_n]$) nas seguintes variáveis:

A. Aim: Snap (Jumps) vs. Flow Aim

- **Métrica:** Distância euclidiana dividida pela janela temporal:

$$V = \frac{(x_n - x_{n-1})^2 + (y_n - y_{n-1})^2}{t_n - t_{n-1}}$$

- **Diferenciação:** Calcule o ângulo θ formado entre três objetos consecutivos (P_{n-1}, P_n, P_{n+1}).
 - Ângulos agudos e fechados com alta aceleração caracterizam **Snap Aim (Jumps)**.
 - Ângulos obtusos e trajetórias circulares contínuas com sliders longos caracterizam **Flow Aim**.

B. Speed vs. Stamina (Tapping)

- **Speed:** Janela temporal pura ($\Delta t = t_n - t_{n-1}$). Sequências onde $\Delta t \leq 62.5$ ms (equivalente a streams em 240+ BPM ou bursts rápidos) geram pontuação no vetor de velocidade de tapping.

- **Stamina:** Quantidade de notas em alta frequência sustentadas sem pausas de descanso ($> 500\text{ ms}$). Aplica-se uma função de decaimento acumulativo (strain decay) que penaliza a fadiga do jogador ao longo de streams longas (ex: picos de 32+ notas contínuas).

C. Finger Control & Complexidade Rítmica

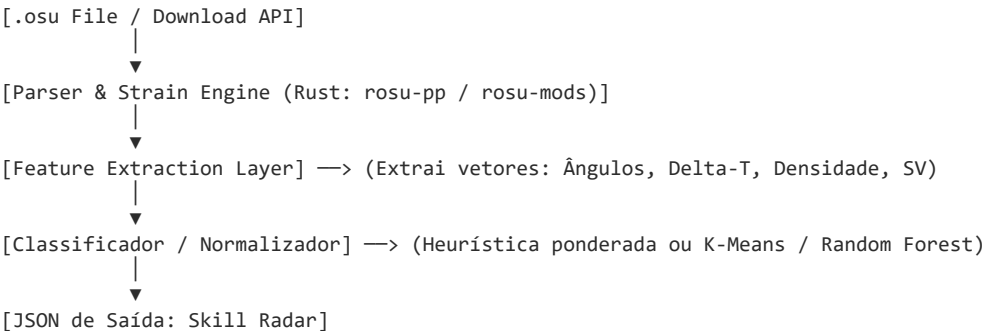
- É o aspecto mais ignorado pelo sistema de PP tradicional, mas fácil de capturar via código.
- **Métrica:** Variância do delta de tempo entre notas sucessivas ($\sigma^2(\Delta t)$). Mapas com mudanças bruscas de snapping (transição de 1/2 para 1/4, introdução de tercinas 1/3 e sextinas 1/6) apresentam alta variância temporal em pequenos intervalos, definindo mapas de controle de dedo e alternância.

D. Reading & Tech (O Gargalo Clássico)

- **Slider Velocity (SV) Dynamics:** Variação abrupta de velocidade de sliders entre seções contíguas.
- **Visual Density / Overlap:** Contagem de objetos visíveis simultaneamente na tela com base no *Approach Rate* (AR). Se múltiplos objetos ocupam o mesmo espaço geométrico com intervalos de tempo dessincronizados do espaçamento visual (violação da relação distância-tempo), a skill de **Reading/Gimmick** atinge seu pico.

2. Arquitetura de Implementação

Não escreva um parser de arquivo .osu do zero em JavaScript puro se você quer performance em escala.



- **Core de Processamento:** Use **Rust** com bindings para Node.js (via *napi-rs*) ou compile para WebAssembly (Wasm) se quiser rodar direto no navegador do cliente sem custo de servidor. O ecossistema *rosu* (especificamente *rosu-pp*) é o padrão da indústria atual para processar beatmaps em microssegundos.
- **Abordagem de Classificação:**
 - **Opção 1: Heurística Direta (Mais simples):** Estabeleça pesos determinísticos para cada métrica com base na distribuição estatística de mapas conhecidos (ex: mapas de DT clássicos definem a régua de Speed).
 - **Opção 2: Machine Learning Não-Supervisionado (Superior):** Extraia o vetor de features de 5.000 mapas ranqueados e aplique **Clustering (K-Means ou DBSCAN)**. O algoritmo agrupará naturalmente os mapas em arquétipos competitivos reais: *Jump Farm*, *Speed/Stamina*, *Tech/Reading*, *Finger Control/Alt* e *Precision*.

3. Onde está o valor prático disso

Essa ferramenta tem valor imediato para dois públicos:

1. **Mappoolers de Torneios:** Em vez de jogar dezenas de mapas para montar um pool equilibrado, o organizador sobe 30 mapas na sua ferramenta e o sistema aponta: *"O slot NM3 está com o dobro de exigência de Finger Control em relação à média do torneio"*.
2. **Recomendação de Treino para Jogadores:** Um jogador insere o perfil dele; o sistema analisa os mapas em que ele dá *choke* (erros por queda de combo) e sugere mapas da comunidade calibrados exatamente no vetor de skill em que ele tem deficiência (ex: *Speed OK, mas baixa tolerância a ângulos agudos em streams*).

User prompt: *É só criar algoritmos robustos para identificar e treinar o algoritmo com mapas já prontos, que tal? Seriam necessários replays ou apenas o arquivo de donwload do mapa?*

Response: Para classificar e extrair os vetores de skill de um beatmap, **apenas o arquivo .osu é suficiente — e é a decisão correta de arquitetura.**

Colocar replays (.osr) no pipeline inicial é uma armadilha de complexidade accidental: infla o custo de armazenamento, exige scraping agressivo e adiciona gargalos de processamento sem necessidade para classificar o mapa.

.osu puro vs. .osu + Replays (.osr)

Dimensão	Apenas .osu	.osu + Replays (.osr)
O que mede	Estímulo / Demanda teórica: geometria, aceleração, janelas de tempo e densidade visual.	Resposta humana: onde os jogadores dão miss, desvio de timing (UR) e comportamento de mira real.

Dimensão	Apenas .osu	.osu + Replays (.osr)
Custo de Dados	Mínimo (~20 KB a 100 KB por arquivo de texto).	Altíssimo (~30 KB a 100 KB por replay , multiplicador de $\mathcal{N}$ plays por mapa).
Viabilidade Técnica	Parsing instantâneo (microsegundos em Rust/WASM). Dá para rodar no próprio browser do usuário.	Requer download de streams binárias comprimidas via API (sujeito a rate limits severos da API v2).
Casos de Uso	Classificação de pools, detecção de arquétipos (Speed, Flow, Tech, Jumps, Alt).	Detecção de <i>choke points</i> específicos e análise biométrica/estilo de grip do jogador.

O perigo de "Treinar" o algoritmo (A armadilha do Ground Truth)

Dizer que vai "treinar" um modelo implica Aprendizado Supervisionado. Para isso funcionar, você precisaria de um dataset rotulado (*ground truth*), o que é inviável no *osu!* por dois motivos:

- 1. **Subjetividade Comunitária:** Não existe consenso matemático sobre onde termina um mapa de "Flow Aim" e começa um mapa de "Tech". Rotulagem manual por humanos introduz ruído e inconsistência no treinamento.
- 2. **Desbalanceamento de Classes:** A grande maioria dos mapas ranqueados gravita em torno de *Jump Farm* e *NM convencional*. Categorias raras como *Reading/Gimmick* ou *High-BPM Poly-rhythm* ficariam sub-representadas, enviesando a rede.

A abordagem de Engenharia correta: 2 Caminhos Superiores

Em vez de aprendizado supervisionado tradicional treinado em rótulos humanos, há dois caminhos consolidados na indústria:

Caminho A: Extração Cinemática Determinística (Recomendado para MVP)

Não use IA. Extraia features puramente físicas do arquivo .osu usando bibliotecas de alto desempenho (como *rosu-pp* ou analisadores vetoriais próprios):

- **Vetor de Ângulos:** Frequência de ângulos agudos ($< 60^\circ$) vs. obtusos ($> 120^\circ$) em saltos com velocidade $> X$ px/ms.
- **Vetor Rítmico:** Entropia de Shannon dos intervalos entre notas (Δt). Se o mapa transita muito entre 1/2, 1/3, 1/4 e 1/6, a entropia rítmica explode $\rightarrow$ **Finger Control**.
- **Vetor de Densidade / Reading:** Quantidade média de objetos concorrentes na janela de tempo de AR dividida pela distância física média entre eles. Se a distância for baixa e o tempo for alto $\rightarrow$ **Reading / Overlap Gimmick**.

Você define equações para gerar uma pontuação normalizada de 0 a 100 em cada eixo. É determinístico, rápido e explicável (você pode apontar exatamente o objeto do mapa que causou a nota alta).

Caminho B: Unsupervised Clustering / Autoencoder (Se quiser usar ML de fato)

Se o objetivo for aplicar Engenharia de IA:

- 1. **Feature Engineering:** Escreva um extrator em Rust/Python que transforme cada .osu em um vetor numérico fixo de 20 a 50 dimensões (médias e variâncias de espaçamento, ângulos, curvatura de sliders, BPM ponderado, variância de SV, etc.).
- 2. **Redução de Dimensionalidade & Agrupamento:** Aplique **PCA / UMAP** seguido de **K-Means / HDBSCAN**.
- 3. **Resultado:** O algoritmo encontrará aglomerados naturais no espaço vetorial. Ao plotar os mapas agrupados, você identificará que o Cluster 1 é naturalmente composto por *Sotarks/Kowari jumps*, o Cluster 2 por *Camellia/Tech*, o Cluster 3 por *Speed streams*, etc.
- 4. **Vantagem:** Nenhum rótulo humano necessário. O algoritmo encontra a taxonomia dos mapas puramente pela distribuição geométrica e temporal dos objetos.

O meio-termo inteligente se você quiser dados de jogadores

Se em algum momento você quiser saber como humanos lidam com o mapa sem carregar o peso dos arquivos .osr, consuma apenas os **metadados das top plays daquele beatmap via API v2**:

- Taxa de $\frac{\text{CountMiss}}{\text{TotalHits}}$ média do top 50: revela se o mapa é punitivo por *choke points*.
- Proporção de notas 100 e 50 em relação aos misses: mapas com muitos 100s mas zero misses indicam complexidade rítmica pura (*Finger Control*), enquanto mapas com alta precisão e misses abruptos indicam picos de *Aim*.

Conclusão direta: Descarte os replays .osr para a classificação do mapa. Foque 100% no parsing das coordenadas e tempos do .osu.

User prompt: Estou pensando em usar uma biblioteca de osu oficial de código aberto em C# (.NET) disponibilizada pelo próprio peppy. Será que isso ajuda e melhora nossa aplicação? Pesquise sobre essa biblioteca caso não conheça ela.

Response: A biblioteca a que você se refere é o núcleo do próprio jogo no repositório *ppy/osu* (especificamente os pacotes NuGet *osu.Game* e *osu.Game.Rulesets.Osu*), frequentemente acompanhados das ferramentas do *ppy/osu-tools* (onde reside o *PerformanceCalculator*).

Usar essa biblioteca oficial traz benefícios claros, mas introduz custos arquiteturais significativos. A avaliação técnica depende de como você planeja estruturar seu backend.

Vantagens: Fidelidade Canônica e Custo Zero de Algoritmo

1. **Paridade Absoluta com o Motor do Jogo:** Arquivos `.osu` possuem regras complexas que implementações caseiras quase sempre erram:
 - **Stacking Leniency:** Notas sobrepostas no mesmo ponto mudam de posição cartesiana dependendo do tempo decorrido. A biblioteca oficial recalcula os offsets exatos (x, y) automaticamente.
 - **Curvaturas de Sliders:** Interpolação precisa de caminhos Bézier, Catmull e arcos circulares para saber onde o cursor precisa estar a cada milissegundo.
2. **Acesso Direto aos Evaluators de Strain:** O pacote `osu.Game.Rulesets.Osu` já decompõe o mapa internamente. Classes como `AimEvaluator`, `SpeedEvaluator` e `FlashlightEvaluator` calculam os picos de esforço físico a cada seção de 400 ms. Você não precisa inventar a matemática de física de mira do zero; basta extrair essas séries temporais de strain já calculadas.

Desvantagens e Riscos Arquiteturais

1. **Acoplamento e Bloat de Dependências:** O pacote `osu.Game` não é uma biblioteca enxuta de parsing matemático; ele é o **jogo inteiro**. Ao importá-lo, seu projeto carrega dependências do `osu-framework`, abstrações de rendering (Silk.NET, Veldrid), áudio e subsistemas de I/O desnecessários para um servidor headless de análise de dados.
2. **Breaking Changes Frequentes:** Como o desenvolvimento do `osu!lazer` é contínuo, classes internas, métodos de avaliação de dificuldade e atributos de hitobjects sofrem refatorações com frequência. Manter as dependências atualizadas exigirá manutenção periódica do seu código de ingestão.
3. **Overhead de Memória e Throughput:** A alocação de memória por beatmap processado é consideravelmente mais alta do que soluções especializadas em linha de comando ou headless, gerando maior pressão sobre o `Garbage Collector` do .NET sob alto tráfego.

A Alternativa Consolidada: `rosu-pp` (Rust)

A comunidade competitiva e serviços de larga escala não costumam rodar `osu.Game` diretamente em produção no backend. O padrão estabelecido é o `rosu-pp`, uma reimplementação em Rust de todo o algoritmo de dificuldade e parsing do `osu!`:

- **Vantagem de Performance:** É ordens de magnitude mais rápido e consome uma fração mínima da RAM.
- **Isolamento Headless:** Contém estritamente a matemática e o parser, sem dependências de engine de jogo ou áudio.
- **Interoperabilidade:** Pode ser compilado para WebAssembly (Wasm) para rodar diretamente no navegador do seu app React, ou exposto como biblioteca nativa para Node.js (via `napi-rs`) ou C# (via FFI).

Critério de Decisão

- **Adote a biblioteca oficial (`osu.Game`) se:** Sua infraestrutura de backend for 100% nativa em **C# (.NET / ASP.NET Core)** e você quiser acesso imediato a todas as mecânicas experimentais do Lazer sem precisar portar nada manualmente. Para mitigar o risco, isole o uso da biblioteca atrás de uma interface (`IBeatmapAnalyzer`) para que o restante da sua aplicação não fique refém dos contratos do jogo.
- **Evite-a se:** Seu backend for em Node.js, Python ou Go, ou se você planejar fazer o processamento no cliente (React). Criar um microserviço isolado em C# apenas para instanciar a biblioteca do peppy adiciona complexidade operacional desnecessária. Nesse cenário, adotar bindings de Rust (`rosu-pp`) é arquiteturalmente superior.

User prompt: E se eu criar essa infraestrutura em uma VPS minha que já utilizo com outras coisas e fazer o processamento dessa biblioteca oficial 100% backend?

Response: Para essa configuração de VPS específica (2 núcleos de vCPU e 2 GB de RAM DDR4), colocar a biblioteca oficial (`osu.Game` em C#) dividindo espaço com outros serviços é uma **decisão de alto risco arquitetural**.

A probabilidade de você enfrentar problemas de estabilidade e travamentos por falta de recursos é crítica.

Os Gargalos Técnicos Dessa Máquina

1. Gargalo Crítico de Memória (OOM Killer):

- **Consumo base da máquina:** O Linux (kernel, systemd, SSH) já consome cerca de 300 MB a 500 MB. Se você já roda outros serviços na VPS (como PostgreSQL, Redis, Nginx ou APIs em Node/Docker), a memória livre real provavelmente fica abaixo de 1 GB.
- **Consumo do `osu.Game`:** O runtime .NET somado à árvore pesada de dependências do jogo (que carrega abstrações de rendering, áudio e structs completas de cada `HitObject`) facilmente consome de 300 MB a mais de 600 MB de *Working Set* durante o parsing e avaliação de um mapa denso.
- **Resultado:** Se dois mapas forem processados simultaneamente ou se uma pool inteira for enviada para análise em lote, a máquina esgotará a RAM física e o **OOM Killer** do Linux será acionado, encerrando o processo do `osu!` ou derrubando as outras aplicações que já rodam no servidor.

2. Saturação de CPU (2 vCPUs):

- O cálculo de *strains* de Aim/Speed exige iteração vetorial sobre milhares de objetos com interpolação geométrica de curvas de sliders. É uma tarefa estritamente *CPU-bound*.
- Rodar esses cálculos em 2 vCPUs compartilhadas vai gerar picos de 100% de uso de processador, degradando a latência de todas as outras aplicações hospedadas na mesma VPS.

3. Armazenamento de 30 GB NVMe:

- Embora o disco seja rápido, 30 GB é um espaço enxuto se você pretende baixar e manter cache de arquivos .osu ou .osz junto com logs do sistema, imagens Docker e banco de dados.

Comparativo de Carga no Backend: osu.Game vs. rosu-pp

Se o processamento for 100% backend nessa máquina modesta de 2 GB de RAM, a escolha da engine dita a viabilidade do projeto:

Métrica	osu.Game (.NET oficial)	rosu-pp (Rust / Binário Nativo)
Consumo de Memória (RAM)	300 MB – 800 MB+ (alto risco de OOM)	5 MB – 20 MB (desprezível na VPS)
Tempo de parsing por mapa	50 ms – 300 ms	< 3 ms
Overhead de Dependências	Carrega subsistemas de engine de jogo	Zero dependências externas (headless puro)
Impacto nas outras aplicações	Alto (disputa agressiva de vCPU e RAM)	Quase nulo

Se você ainda quiser usar a biblioteca oficial em C#

Se a paridade oficial do .NET for um requisito inegociável para você, adote as seguintes travas de infraestrutura:

1. **Fila com Concorrência Estrita ($N = 1$):** Utilize BullMQ, RabbitMQ ou uma fila interna via Channel<T> no ASP.NET configurada para processar rigorosamente **um beatmap por vez**. Nunca processe requisições de análise em paralelo nessa máquina.
2. **Isolamento via Docker com Limites de Memória:** Configure limites estritos no docker-compose para o container do worker:

```
deploy:
  resources:
    limits:
      cpus: '1.0'
      memory: 700M
```

Isso garante que, caso o cálculo exceda a memória, apenas o container do worker seja reiniciado, sem derrubar seus outros serviços de produção.

3. **Swap em NVMe:** Configure ao menos 2 GB de arquivo Swap no disco NVMe para mitigar picos transitórios de alocação de memória do garbage collector do .NET.

Conclusão Direta

Para essa VPS de 2 GB de RAM e 2 vCPUs que já está em uso, a solução tecnicamente correta é compilar um worker simples em **Rust (rosu-pp)** e expor um micro-endpoint HTTP/gRPC local. Você terá um processamento centenas de vezes mais rápido e manterá seus outros serviços em produção completamente estáveis.

User prompt: Isso significa que mesmo se eu quiser instalar esse sistema em um app exe no meu próprio sistema, ainda sim analisar uma pool inteira de uma vez seria semelhante a abrir uma pool inteira pra jogar ao mesmo tempo?

Response: **Não.** Essa analogia é um exagero conceitual. Analisar uma pool via código não é nem de longe equivalente a abrir dezenas de instâncias do jogo para jogar ao mesmo tempo.

A restrição levantada anteriormente dizia respeito exclusivamente àquela **VPS de 2 GB de RAM compartilhada com outros serviços de produção**, onde qualquer pico de 500 MB causa pânico no sistema operacional. Em uma máquina desktop comum, a realidade é completamente diferente.

O que muda entre "Jogar" e "Analisar via Código (Headless)"

Quando você abre o osu! para jogar, a máquina executa:

- Inicialização da engine gráfica (contexto Vulkan/DirectX/OpenGL e pipelines de shader).
- Carregamento de skins, texturas, vídeos de fundo e decodificação contínua de áudio.
- Game loop rodando a até 1000 Hz, capturando entradas de teclado/tablet e renderizando quadros em tempo real.

Quando você executa a análise via código (como faz o osu-tools do peppy):

- Nenhuma janela é criada e nenhum driver de áudio ou vídeo é iniciado.
- O sistema apenas lê o arquivo .osu, mapeia as coordenadas e tempos em memória, roda o cálculo de física de cada objeto e descarta os dados assim que o resultado sai.
- O processo leva de **50 a 150 milissegundos por beatmap**.

Processamento Local (.exe) na sua Máquina

Se você desenvolver um aplicativo desktop local em C# (.NET) para rodar na sua própria máquina (que costuma ter 8 GB, 16 GB ou mais de RAM e CPUs desktop de alta performance):

1. **Uma pool inteira (20 a 30 mapas) processa em 2 a 4 segundos.**
2. **Consumo desprezível no Desktop:** O processo usará cerca de 200 MB a 400 MB de RAM temporariamente e liberará em seguida via Garbage Collector. Para um PC de mesa, isso não causa nenhum impacto perceptível.
3. **Controle de Fluxo Simples:** Você não precisa processar todos os mapas em paralelo saturando todas as threads. Um simples loop sequencial (foreach) já resolve uma pool completa quase que instantaneamente:

```
// Exemplo em C# consumindo osu.Game de forma headless e sequencial
foreach (var mapPath in poolBeatmaps)
{
    var beatmap = BeatmapDecoder.Decode(mapPath);
    var difficultyCalculator = new OsuDifficultyCalculator(ruleset, beatmap);
    var attributes = difficultyCalculator.Calculate();
    // Extrai strains e salva os dados...
}
```

O Veredito Prático

- **Na VPS de 2 GB:** Rodar a biblioteca oficial do peppy em background é arriscado porque o ambiente é restrito e divide memória com banco de dados e outros sistemas web.
- **No seu PC local (.exe):** É uma excelente escolha de prototipagem. Você terá paridade total com a matemática oficial do jogo, zero risco de travar seu sistema operacional e facilidade para iterar rapidamente na lógica de detecção de *skills*.

User prompt: *ainda sim é melhor o rust ou c#*

Response: Tecnicamente e arquiteturalmente, **Rust é superior** para este projeto.

C# só empata ou ganha em um único cenário: conveniência pessoal caso você já domine a linguagem e queira evitar a curva de aprendizado do ecossistema Rust.

Abaixo está a análise comparativa direta entre os dois caminhos considerando que sua interface final será em **React**.

Por que Rust (rosu-pp) é a melhor escolha técnica

1. A cartada do WebAssembly (WASM) no React (Custo Zero de Servidor):

- O `rosu-pp` pode ser compilado diretamente para **WebAssembly**.
- **Impacto prático:** Você importa o módulo WASM no seu frontend React. Quando o usuário arrasta um arquivo `.osu` ou uma pasta de pool inteira para o seu site, **o navegador do próprio usuário processa a física do mapa em microssegundos**.
- Você elimina a necessidade de fazer upload de arquivos `.osu` pesados para a sua VPS e reduz o custo computacional do seu servidor a praticamente zero. C# até possui suporte a WASM via Blazor, mas gera bundles massivos e lentos para inicializar no browser.

2. Isolamento de Domínio (Headless Real):

- A biblioteca em Rust foi desenhada especificamente para uma única tarefa: calcular dificuldade, strains e performance sem nenhuma dependência gráfica ou de áudio.
- A biblioteca oficial em C# (`osu.Game`) carrega o ecossistema inteiro do jogo. Qualquer script headless em C# ainda carrega classes do framework de rendering, áudio e structs complexas de interface.

3. Portabilidade Extrema:

- Se você mantiver o processamento no backend da sua VPS, um binário compilado em Rust consome **menos de 15 MB de RAM**, roda a centenas de mapas por segundo e pode ser chamado via Node.js (com bindings `napi-rs`), Python ou um micro-endpoint HTTP/gRPC.
- Se você quiser empacotar como um app desktop executável (`.exe`), o framework **Tauri** permite usar seu frontend em **React** com o backend em **Rust**, gerando um executável de 10 MB que consome quase nada de memória (ao contrário do Electron ou de um app C# pesado).

Onde C# (.NET) ainda tem vantagem

1. Garantia Canônica Imediata:

- Se o peppy e a equipe do Lazer alterarem o algoritmo de cálculo de Star Rating ou mudarem a fórmula de *strains* de sliders na branch principal, o pacote NuGet oficial em C# reflete isso de forma nativa.
- No Rust, você depende da atualização do `rosu-pp` (embora os mantenedores do projeto costumem portar as mudanças do Lazer com extrema rapidez, frequentemente em questão de dias).

2. Menor Custo Cognitivo (se você já programa em .NET):

- Se você nunca encostou em Rust, lidar com o *borrow checker*, *lifetimes* e manipulação de tipos em baixo nível pode atrasar a entrega do MVP.
- Em C#, instanciar um `OsuDifficultyCalculator` e consumir um objeto LINQ é trivial para quem já conhece a plataforma.

Quadro Comparativo Direto

Critério	Rust (rosu-pp)	C# (osu.Game / osu-tools)
Execução no Browser (WASM)	Excelente (binários leves e nativos)	Ruim (runtime .NET pesado para web)
Consumo de Recursos (VPS/Local)	Mínimo (~10 MB RAM, latência < 2ms)	Médio/Alto (200 MB – 500 MB RAM)
Integração com React	Nativa via WASM, Tauri ou Napi-rs	Requer API REST intermediária
Curva de Aprendizado	Alta (se não souber Rust)	Baixa / Média (.NET convencional)
Manutenibilidade no osu!	Biblioteca enxuta e focada em dados	Acoplada ao ciclo de vida do jogo

Decisão Prática

Se o objetivo é construir uma aplicação moderna com **React**:

- **Vá de Rust:** Você compila a lógica de extração para WebAssembly e roda tudo direto no frontend (ou empacota com Tauri caso queira um .exe). Isso resolve o problema de infraestrutura, não sobrecarrega sua VPS e entrega a melhor arquitetura possível.
- **Fique no C#:** Apenas se o tempo de entrega do projeto for urgente, você não tiver disposição para aprender a sintaxe de Rust agora e preferir montar uma API REST em ASP.NET Core rodando na sua máquina local ou em um container restrito.

User prompt: Eu programo 100% com o antigravity.

Response: Seja essa referência a um fluxo de desenvolvimento 100% orientado a agentes de IA ou ao clássico ecossistema Python (import antigravity), isso altera a equação de decisão técnica de formas bem específicas:

Cenário 1: Você delega a escrita do código para IA / Agentes

Se a barreira sintática e o custo de implementação humana foram eliminados por automação, **Rust se torna a escolha definitiva**.

- **O compilador como corretor determinístico:** O maior problema de gerar código via IA em linguagens fracamente tipadas ou permissivas (como Python ou JavaScript) são os erros silenciosos em tempo de execução. Em Rust, o rustc atua como um validador implacável. Se o código gerado passar pelo borrow checker e compilar, a probabilidade de falhas de memória, concorrência ou crashes em produção na sua VPS é quase nula.
- **O escopo de código é minúsculo:** Você não precisa pedir para a IA criar um analisador físico do zero. A biblioteca rosu-pp já entrega a matemática pronta. O que a IA precisará gerar são apenas 40 a 60 linhas de glue code para receber o arquivo .osu, extrair os vetores de strain e retornar o JSON para o React (ou compilar para WASM).
- **Onde a IA falha em Rust:** Modelos de linguagem frequentemente entram em loops tentando resolver erros de tempo de vida (lifetimes) ou referências cíclicas em Rust. Mantenha as estruturas simples: passe dados por valor ou clones baratos e evite referências complexas entre structs.

Cenário 2: Você se refere a programar em Python (import antigravity)

Se a afirmação significa que a sua stack padrão é Python, há impactos diretos na arquitetura:

- **Existe binding oficial:** Você não precisa sofrer com a lentidão padrão do interpretador Python para cálculo numérico. A comunidade mantém o pacote rosu-pp-py (escrito em Rust com interface CPython), que processa mapas com a mesma velocidade nativa.
- **A perda do WebAssembly no frontend:** Rodar Python no navegador do usuário (via Pyodide/Wasm) é inviável para uma aplicação web ágil (o bundle inicial ultrapassa 30 MB apenas para carregar o runtime).
- **Carga concentrada na VPS:** Toda a computação terá que ocorrer obrigatoriamente no seu backend. Isso significa que sua VPS de 2 GB de RAM terá que lidar com o upload contínuo de arquivos .osu, enfileiramento e respostas HTTP, em vez de deixar o navegador do cliente fazer o trabalho pesado de graça.

Veredito Prático

Se a codificação manual não é mais o gargalo, a decisão deixa de ser sobre qual linguagem você escreve mais rápido e passa a ser puramente sobre **eficiência do artefato final**:

1. Peça ao seu assistente de código para gerar um módulo em **Rust com wasm-pack** expondo uma função analyze_beatmap(osu_file_bytes) -> JsValue.
2. Importe esse binário .wasm diretamente no seu projeto **React**.
3. Deixe o navegador de quem estiver usando a aplicação calcular as skills instantaneamente, mantendo a sua VPS com custo zero de processamento e totalmente livre para os outros sistemas que já rodam nela.

User prompt: é vibe coding dentro do próprio programa do antigravity com a escrita de código e validação/criação de testes em tempo real.

Response: Com um fluxo de vibe coding apoiado por um ciclo contínuo de testes e validação em tempo real (TDD agêntico), a fricção sintática desaparece. O seu papel deixa de ser escrever sintaxe e passa a ser **arquitetura, restrição de escopo e especificação de testes**.

Nesse cenário, **Rust é ainda mais vantajoso**, mas o seu agente de IA cairá em armadilhas específicas do ecossistema de *osu!* se você não impor as restrições corretas.

Por que o loop agêntico funciona melhor com Rust aqui

- 1. **Feedback Determinístico do Compilador:** O `rustc` cospe erros estruturados e altamente detalhados no terminal. Quando o agente erra tipos, lifetimes ou contratos de API, o compilador entrega o diagnóstico exato, permitindo que o loop de autocorreção do agente resolva a falha em poucas iterações sem intervenção manual.
- 2. **Ausência de Erros Silenciosos:** Em Python ou JavaScript, agentes frequentemente passam em testes com tipos errados ou `null pointers` que só explodem em produção. Em Rust, teste que compila e roda com `cargo test` garante estabilidade operacional total.

As 3 Armadilhas do Agente de IA neste projeto (e como evitá-las)

Ao delegar a implementação para o agente, imponha as seguintes regras no prompt inicial para não deixá-lo entrar em loops improdutivos:

1. Impedir a reinvenção do parser

- **O risco:** O agente tentará escrever um parser de `.osu` regex/string do zero e inventar a física de interpolação de sliders.
- **A diretiva:** Exija explicitamente o uso da crate `rosu-pp` para todo o parsing e cálculo de strains (`Beatmap::from_bytes, Difficulty`). O trabalho do agente deve ser estritamente mapear a saída do `rosu-pp` para as suas categorias de skill (Aim, Speed, Precision, Reading).

2. Evitar testes unitários com "mapas falsos"

- **O risco:** O agente vai criar strings fictícias simulando arquivos `.osu` nos testes unitários. Essas strings não refletem o timing, o stacking nem as curvas reais do jogo, gerando falsos positivos nos testes.
- **A diretiva:** Baixe 3 arquivos `.osu` reais representativos (ex: um mapa puro de Jumps/Aim, um de Speed/DT e um de Tech/Reading) e coloque-os em uma pasta `tests/fixtures/`. Instrua o agente a rodar os testes de validação exclusivamente contra esses arquivos reais, verificando se o vetor de skill gerado reflete a categoria esperada de cada mapa.

3. Gargalo de WASM bloqueando a UI no React

- **O risco:** Se o agente compilar para WASM e chamar a função síncrona diretamente no componente React, a interface vai travar por frações de segundo ao processar pools inteiras (20+ mapas de uma vez).
- **A diretiva:** Ordene que a integração do módulo WASM no frontend seja encapsulada dentro de um **Web Worker**. O usuário faz o drag-and-drop da pool, a UI continua a 60 FPS com animações suaves e o Worker devolve os resultados das skills via `postMessage`.

O Roteiro de Prompts para o seu Ambiente

Execute em duas fases desacopladas para evitar context window bloat no agente:

Fase 1: Core de Extração em Rust (Headless + Testes)

"Crie uma biblioteca em Rust chamada `osu_skill_extractor` configurada com `wasm-pack`. Adicione `rosu-pp` e `serde`. Crie uma função que receba os bytes de um arquivo `.osu` e retorne uma struct contendo métricas normalizadas (0 a 100) para Aim, Speed, Stamina e Rhythm Complexity. Na pasta `tests/fixtures`, use os 3 arquivos `.osu` reais fornecidos para validar os cálculos em testes de integração via `cargo test`."

Fase 2: Módulo React / Vite

"Configure um projeto React com Vite e TypeScript. Importe o pacote WASM gerado na Fase 1 utilizando um Web Worker. Construa uma interface de drag-and-drop para arquivos `.osu` ou pastas compactadas `.zip/.osz`. Exiba o resultado agregado da pool através de gráficos de radar (Stat Hexagon) e barras comparativas por slot de mod."

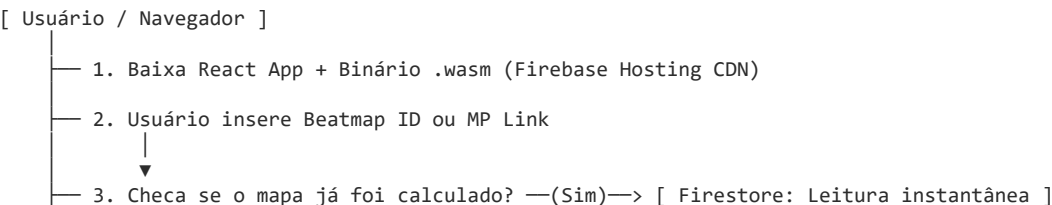
Essa divisão mantém o agente focado em pipelines determinísticos e entrega uma aplicação que roda 100% no navegador do usuário, com custo zero de manutenção na sua VPS.

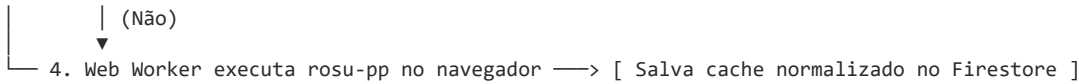
User prompt: Eu mencionei a VPS, mas tenho projetos no Firebase Hosting e firestore também

Response: O uso de **Firebase Hosting e Firestore**, combinado com o processamento em **Rust/WASM**, elimina por completo a necessidade de usar sua VPS para esse projeto.

Essa mudança transforma a arquitetura de uma infraestrutura tradicional (servidor dedicado processando arquivos pesados) para um modelo **Serverless Jamstack com computação na borda (no cliente)**, com custo operacional próximo de zero.

Como a arquitetura se divide com Firebase





1. Firebase Hosting: Distribuição de WASM em CDN Global

- **Vantagem direta:** O Firebase Hosting serve arquivos estáticos via Fastly/Google CDN. O bundle do seu app React e o binário .wasm (que compilado com wasm-opt fica na faixa de 1 MB a 3 MB) são baixados instantaneamente pelo navegador do usuário.
- **MIME Types automáticos:** O Firebase Hosting já serve a extensão .wasm com o header application/wasm correto e compressão Brotli/Gzip nativa, garantindo que o browser faça o *streaming compilation* da engine de Rust instantaneamente.

2. Firestore: Cache Determinístico de Beatmaps e Matches

Como um beatmap do *osu!* é estático (o mapa não muda a física depois de ranqueado), você **nunca deve calcular o mesmo mapa duas vezes na vida do seu aplicativo**.

Modelagem de Coleções no Firestore:

- **Coleção beatmaps (Document ID = beatmap_id):** Guarda apenas o vetor de skills extraído, não a geometria inteira do mapa.

```

{
  "beatmap_id": 5337775,
  "title": "NIGHTMARE INVITATION",
  "version": "eringiRa's Insane",
  "bpm": 210,
  "drain_time": 146,
  "skills": {
    "aim_snap": 68.4,
    "flow_aim": 42.1,
    "speed": 81.0,
    "stamina": 54.2,
    "rhythm_complexity": 72.8,
    "reading_density": 60.5
  },
  "calculated_at": 1781834950
}
  
```

- **Coleção matches (Document ID = match_id):** Guarda o histórico consolidado de auditoria de torneios extraído do link do Bancho (121918084).

```

{
  "match_id": 121918084,
  "tournament": "6WC26",
  "team_red": "Brazil B",
  "team_blue": "United States B",
  "final_score": { "red": 1, "blue": 6 },
  "rounds": [ ... ],
  "mvp_player_id": 19499604
}
  
```

3. A Armadilha de Segurança e o Problema de CORS

Ao usar Firebase direto no cliente, dois gargalos práticos surgem:

1. **Bloqueio de CORS ao baixar o .osu:** O navegador do usuário não consegue dar `fetch("https://osu.ppy.sh/osu/:id")` (`https://osu.ppy.sh/osu/:id`) diretamente porque a infraestrutura da ppy bloqueia requisições Cross-Origin de outros domínios web.
 - **Solução:** Você precisa de uma rota para intermediar o download do arquivo de texto bruto.
 - **Como resolver sem a VPS:** Crie uma **Cloud Function** (no próprio ecossistema do Firebase) ou um **Cloudflare Worker** simples que apenas bate no Bancho/Mirror (como Chimu, Nerinyan ou Sayobot), baixa o .osu e entrega com o cabeçalho `Access-Control-Allow-Origin: *`.
2. **Integridade dos Dados no Firestore (Security Rules):** Se o cálculo roda no cliente e o cliente escreve direto no Firestore, um usuário mal-intencionado pode abrir o DevTools e salvar que determinado mapa tem `aim_snap: 999.0`.

- **Solução via Security Rules:**

```

rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /beatmaps/{beatmapId} {
      allow read: if true;
      // Permite escrita apenas se as métricas estiverem dentro dos limites matemáticos válidos
      allow create: if request.resource.data.skills.aim_snap >= 0
        && request.resource.data.skills.aim_snap <= 100
        && request.auth != null; // ou restrito via Cloud Function de ingestão
    }
  }
}
  
```

```
}
}
```

- **Abordagem ideal:** Deixe o Firestore em modo de **apenas leitura** (`read: true`) **para o público** e faça a escrita inicial do mapa passar por uma Cloud Function leve que valida o processamento.

Veredito da Stack

Essa estrutura com Firebase é **muito superior à arquitetura da VPS**:

- **VPS (2 vCPUs / 2GB):** Fica completamente livre para seus outros projetos existentes, sem risco de quedas por consumo de memória ou concorrência de CPU.
- **Firebase Hosting:** Distribui o app React e o WASM com performance de borda.
- **Firestore:** Funciona como cache compartilhado entre todos os usuários do torneio (se um capitão de time analisou o pool da 6WC, todos os outros jogadores da comunidade recebem as skills dos mapas instantaneamente via cache).

User prompt: E se eu usar a própria api v2 pra consultar os mapas sem necessariamente invocar url?

Response: A API v2 oficial do *osu!* **não resolve o problema** se o seu objetivo for criar métricas reais de *skills* personalizadas (como *Reading*, *Tech*, *Finger Control* e *Flow Aim*).

A premissa falha porque a API v2 **não expõe os dados brutos do mapa** (as notas, tempos e coordenadas cartesianas), apenas metadados agregados e atributos pré-calculados pela engine padrão.

O que a API v2 entrega vs. O que você precisa

O endpoint analítico da API v2 (POST `/api/v2/beatmaps/{id}/attributes`) retorna apenas o cálculo que a ppy já faz para o Star Rating oficial:

```
{
  "attributes": {
    "star_rating": 6.42,
    "max_combo": 1240,
    "aim_difficulty": 3.15,
    "speed_difficulty": 2.89,
    "flashlight_difficulty": 0.0,
    "slider_factor": 0.94,
    "speed_note_count": 412.0,
    "approach_rate": 9.6,
    "overall_difficulty": 9.2
  }
}
```

- **O que dá para fazer só com a API v2:** Gráficos simplistas comparando apenas o **Aim tradicional vs. Speed tradicional** oficial do jogo.
- **O que é matematicamente impossível calcular pela API v2:**
 - **Finger Control:** Você precisa dos timestamps (t) de cada nota para calcular a entropia dos intervalos (Δt). A API não fornece timestamps individuais.
 - **Snap vs. Flow Aim:** Você precisa das posições (x, y) para calcular a curvatura e aceleração angular dos saltos. A API não entrega coordenadas.
 - **Reading / Gimmick:** Você precisa da sobreposição geométrica em tela dentro da janela de AR. A API entrega apenas o valor escalar de AR (ex: 9.6).
 - **Complexidade de Sliders (Tech):** A API entrega apenas um multiplicador genérico (`slider_factor`).

Os Gargalos de Infraestrutura da API v2

Optar por consultar a API v2 em vez de baixar o arquivo `.osu` traz restrições adicionais:

1. Autenticação Obrigatória (OAuth2 Client Credentials):

- Você precisa registrar uma aplicação no *osu!*, obter `client_id` e `client_secret`, e implementar fluxo de renovação de Bearer token no backend.
- Se você colocar essas credenciais direto no React, expõe suas chaves privadas. Você seria forçado a ter um intermediário (Cloud Function / backend).

2. Rate Limits Severos:

- A API oficial impõe limites estritos (cerca de 60 a 120 requisições por minuto).
- Se um torneio submeter uma pool inteira de 30 mapas ou múltiplos usuários analisarem lobbies ao mesmo tempo, sua aplicação tomará 429 Too Many Requests.

3. Ausência de download direto de arquivos `.osu` na v2:

- A API v2 só disponibiliza download de pacotes compactados .osz via GET `/api/v2/beatmapsets/{id}/download`, o que exige sessão de usuário logado (OAuth com escopo de identificação) e gera downloads pesados com áudio e vídeo desnecessários.

Como resolver sem complicar a arquitetura

O arquivo de texto puro .osu reside estaticamente em um endereço público e leve:

URL Direta: `[https://osu.ppy.sh/osu/](https://osu.ppy.sh/osu/){beatmap_id}`

- **Peso:** 20 KB a 80 KB de texto puro (sem mp3, sem background, sem vídeo).
- **Velocidade:** O download do texto leva menos de 100 ms.
- **Solução para o CORS:** No Firebase, você cria uma **Cloud Function de 15 linhas** (ou um Cloudflare Worker gratuito) que apenas faz:

```
// Proxy minimalista para evitar bloqueio de CORS no navegador
export const getOsuFile = onRequest(async (req, res) => {
  const { id } = req.query;
  const response = await fetch(`https://osu.ppy.sh/osu/${id}`);
  const text = await response.text();

  res.set("Access-Control-Allow-Origin", "*");
  res.set("Content-Type", "text/plain");
  res.send(text);
});
```

Veredito

Não tente contornar o download do arquivo .osu usando a API v2. A API não contém a matemática do mapa necessária para criar algoritmos de skills.

A arquitetura correta é: **baixe o .osu puro via proxy simples → processe a física localmente no navegador via Rust/WASM → salve os atributos resultantes no Firestore.**

User prompt: Entendi. Eu eu realmente precisaria baixar os mapas. Mas e se a princípio eu criasse um sistema de "selecione o arquivo/pasta dos mapas e importe"?

Response: Essa é a **melhor decisão de engenharia possível para o MVP**.

Ao delegar a origem do arquivo para o sistema de arquivos local do usuário, você elimina instantaneamente 80% da complexidade acidental que estávamos discutindo:

1. **Adeus ao CORS e Proxies:** Você não precisa de Cloud Functions, Cloudflare Workers ou VPS para intermediar downloads da ppy.
2. **Risco zero de Rate Limit:** Nenhuma requisição à API oficial ou a servidores de download é feita.
3. **Custo e Infraestrutura Zero:** Roda 100% no navegador do usuário via File API + Rust/WASM, sem gastar um centavo de banda ou CPU no seu Firebase/VPS.
4. **Alinhamento com o Comportamento Real do Jogador:** Mappoolers e jogadores de torneio **já baixam o mappool pack** (.zip ou .osz) para treinar. O arquivo já está no disco deles.

Como estruturar o fluxo de importação no React

Existem dois formatos que os usuários costumam ter em mãos: arquivos .osu avulsos ou o pacote do pool (.osz / .zip).

```
[ Usuário solta .osz / .zip / .osu ]
      |
      v
[ fflate (Descompressor leve em JS) ] —> Extrai apenas arquivos com extensão .osu
      |
      v
[ Web Worker + WASM (rosu-pp) ] —————> Processa cada ArrayBuffer em microssegundos
      |
      v
[ Estado React (Zustand / Redux) ] —————> Alimenta os gráficos de radar e métricas
```

1. Input de Múltiplos Arquivos e Pastas

No React, você pode permitir tanto o upload clássico de arquivos múltiplos quanto de pastas completas usando atributos nativos:

```
<input
  type="file"
  multiple
  webkitdirectory="true" // Permite selecionar uma pasta inteira de uma vez
  accept=".osu,.osz,.zip"
  onChange={handleFiles}
/>
```

2. Extração de .osz direto na memória (Client-side Unzip)

Se o usuário arrastar um .osz (que nada mais é do que um .zip renomeado):

- Não suba para nenhum servidor. Use uma biblioteca ultra-leve e assíncrona como `fflate` (muito mais rápida e menor que `jszip`).
- Isole e leia **apenas os arquivos .osu** dentro do pacote, ignorando completamente os arquivos .mp3, .wav, vídeos e imagens de fundo para não gastar memória do navegador.
- Converta o texto do .osu para um `Uint8Array` (bytes) e passe direto para a função exportada pelo seu binário WASM.

Cuidados e Travas de UX necessárias

Para essa abordagem não falhar na prática, adicione duas validações no código:

1. **Bloqueio da pasta Songs inteira:** Se o usuário tentar selecionar a pasta raiz do *osu!* dele com 50.000 mapas, o navegador pode congelar na indexação dos arquivos. Coloque um limitador (ex: *"Máximo de 50 mapas por lote"* ou avise se o diretório tiver mais de *X* arquivos).
2. **Filtro de Dificuldade:** Um pacote .osz costuma conter todas as dificuldades de um set (Easy, Normal, Hard, Insane, Extra). Em torneios, apenas a dificuldade mais alta costuma fazer parte do pool. O sistema deve listar as dificuldades encontradas no pacote e permitir que o usuário selecione/filtre quais entram na análise da pool.

Veredito

Começar por essa ferramenta local de importação entrega um produto funcional **imediatamente**. Você valida a aceitação do seu sistema de cálculo de *skills* junto à comunidade sem depender de infraestrutura externa, autenticação ou autorizações da ppy.

Assim que os algoritmos de radar estiverem consolidados e a comunidade validar a utilidade das métricas, integrar links online passa a ser apenas uma funcionalidade incremental secundária.

User prompt: Mantendo então essa arquitetura client side é possível?

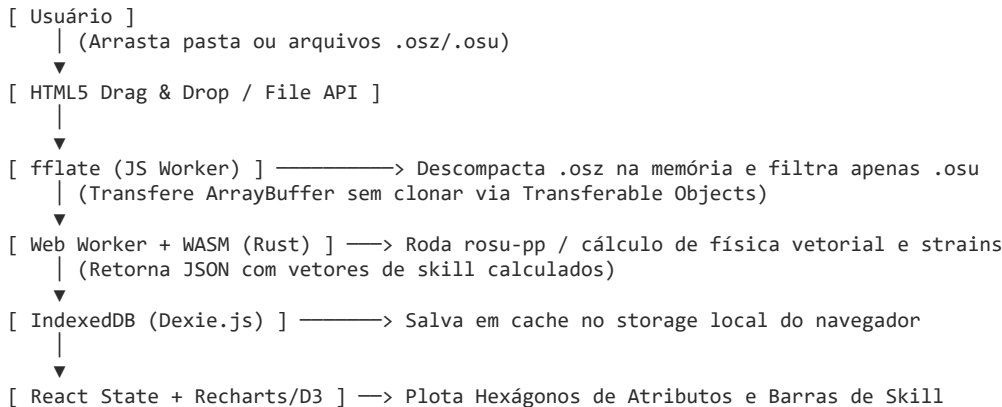
Response: Sim, é **100% viável**.

Essa abordagem transforma sua aplicação em uma **Single Page Application (SPA) autossuficiente e offline-first**.

Ao eliminar qualquer dependência de backend (VPS, Docker, Cloud Functions ou APIs proprietárias), o navegador do usuário assume todo o ciclo de vida do dado: leitura de disco, descompressão, parsing matemático e renderização visual.

A Pipeline 100% Client-Side

A arquitetura completa roda dentro do runtime do browser:



Componentes Técnicos Essenciais

1. Persistência Local sem Banco de Dados (IndexedDB)

Sem backend, você não precisa de Firestore para persistir dados. Use o **IndexedDB** do próprio navegador (com wrappers como `Dexie.js` ou `idb`).

- O usuário importa a pool uma vez.
- O cálculo das skills é processado e indexado no banco de dados local do navegador.
- Quando o usuário reabrir o site ou atualizar a página, os dados já estarão lá instantaneamente sem recalculá-los.

2. Processamento sem Bloqueio de UI (Web Worker + Transferable Objects)

Processar 30 arquivos .osu em JavaScript na thread principal causaria congelamento de micro-segundos na interface do React.

- O arquivo .osu bruto é lido como um `ArrayBuffer`.

- Esse buffer é transferido (e não copiado) para um **Web Worker** via:

```
worker.postMessage({ type: 'ANALYZE_MAP', buffer }, [buffer]);
```

- O Worker carrega o módulo WebAssembly, processa os cálculos cinemáticos em Rust e devolve apenas a struct final com os números calculados (Aim, Speed, Tech, Reading, Stamina).
- A interface React permanece fixa a 60/120 FPS com transições suaves.

3. Compartilhamento entre Usuários (Export / Import JSON)

Se você não tiver um banco centralizado e um organizador de torneio quiser compartilhar a análise do pool com os capitães dos times:

- Basta criar um botão **"Exportar Pool (.json)"**.
- O outro usuário apenas abre a aplicação e clica em **"Importar Pool"**. O JSON carrega o estado pré-calculado no IndexedDB dele imediatamente.

Onde Fica o Limite Dessa Arquitetura

O modelo client-side resolve **completamente** a análise de mapas e pools. Contudo, fique atento ao único ponto de atenção caso decida retomar a ideia dos links de partidas multijogador (*MP Links* do Bancho):

- **A barreira de CORS do Bancho:** O navegador do usuário **não pode** fazer `fetch("https://osu.ppy.sh/community/matches/121918084")` diretamente, porque o servidor da ppy não envia o cabeçalho `Access-Control-Allow-Origin: *`.
- **Solução client-side para isso (se for usar):** Permitir que o usuário cole o texto bruto do chat/histórico do MP Link ou carregue o JSON gerado diretamente pelo Bancho via input de texto.

Veredito Prático

Manter o processamento client-side com **React + WebAssembly (Rust) + IndexedDB**:

1. **Zera seu custo operacional:** Hospedagem gratuita no Firebase Hosting, GitHub Pages, Vercel ou Cloudflare Pages.
2. **Elimina riscos de infraestrutura:** Nenhuma VPS para monitorar, sem problemas com falta de RAM e zero risco de tomar ban por rate-limit de IP em requisições de download.
3. **Escalabilidade infinita:** O custo computacional de 1 usuário ou de 10.000 usuários é idêntico para você, pois cada cliente usa o próprio processador para calcular os mapas.

User prompt: mas não pesaria o navegador do usuário?

Response: **Não**. Se a implementação for feita corretamente, processar uma pool inteira de 30 a 40 mapas consome **menos de 50 MB de RAM** e leva **menos de 100 milissegundos de CPU**.

Uma aba comum do YouTube ou Twitter/X consome entre 400 MB e 1,2 GB de RAM. Comparado a isso, o processamento de uma pool em WebAssembly é imperceptível para o navegador.

O navegador só pesaria se você cometesse erros básicos de gerenciamento de memória e concorrência no frontend. Abaixo estão os números reais e os três mecanismos que garantem que a aplicação permaneça leve:

A Ordem de Grandeza Real dos Dados

1. **Volume de Texto:** Um arquivo .osu tem entre 20 KB e 80 KB. Uma pool inteira de 35 mapas soma menos de **3 MB de texto bruto**.
2. **Tempo de Execução em WASM:** O `rosu-pp` compilado para WebAssembly executa a cinemática de um mapa em **1 a 3 milissegundos**.

Tempo total: $35 \text{ mapas} \times 2 \text{ ms} \approx 70 \text{ ms}$

O cálculo termina antes mesmo do usuário perceber uma barra de progresso.

3. **Pegada de Memória:** O binário WASM precisa de apenas uma página de memória linear inicial (~16 MB a 32 MB), liberada após a computação.

O que impede o navegador de travar

Para garantir que o app rode a 60/120 FPS sem nenhum micro-engasgo (*jank*), a arquitetura precisa seguir três regras:

1. Web Worker isola a UI (Off-Main-Thread)

Se você rodar qualquer loop pesado na thread principal do JavaScript, a interface congela e as animações do React travam.

- Ao mover o binário WASM para um **Web Worker**, o cálculo roda em uma thread secundária do sistema operacional.
- A thread principal do React continua 100% dedicada a renderizar os componentes, transições e gráficos sem nenhuma perda de quadros.

2. Extração Seletiva de Streams (Descarte de Áudio e Vídeo)

O maior perigo para a memória não é o arquivo `.osu`, mas sim o pacote `.osz/.zip` do pool, que costuma pesar entre 150 MB e 400 MB devido a músicas (`.mp3`), vídeos e imagens de fundo.

- **O erro:** Usar uma biblioteca que carrega o `.zip` inteiro como um `Blob` na memória RAM do navegador para depois extrair.
- **A solução:** Usar `fflate` em modo de *streaming*. O leitor varre o cabeçalho do arquivo compactado, extrai **exclusivamente os arquivos que terminam com `.osu`** e descarta todos os bytes de áudio e imagem sem sequer alocá-los na memória. Os 300 MB do `.zip` viram apenas 3 MB úteis na RAM.

3. Uso de Transferable Objects

Ao enviar os bytes do arquivo `.osu` da thread principal para o Web Worker, passe o `ArrayBuffer` como um *Transferable Object*:

```
// Transfere a posse da memória em vez de clonar/duplicar os bytes na RAM
worker.postMessage({ buffer }, [buffer]);
```

Isso tem custo zero de cópia de memória ($O(1)$), transferindo o ponteiro diretamente entre threads sem gerar lixo para o *Garbage Collector*.

Onde poderia dar problema (e como prevenir)

O único cenário em que o navegador sofreria lentidão seria se o usuário selecionasse acidentalmente a pasta raiz do jogo (`osu!/Songs`) contendo dezenas de milhares de mapas.

A proteção técnica para isso é uma trava simples na camada de entrada:

- **Batch Cap:** Limite a leitura de pacotes a um teto razoável para torneios (ex: máximo de 60 arquivos `.osu` por lote).
- Se a contagem ultrapassar o limite, aborte o parsing imediatamente e exiba um aviso na tela.

User prompt: A princípio inclusive, para treinar inicialmente o nosso sistema dentro do Antigravity, vou procurar playlists inteiras de 30 a 100 beatmaps se disser para o antigravity "Isso aqui é uma pasta só de mapas de jumps" ou "isso aqui é uma pasta só de mapas de streams". Seria uma boa ideia?

Response: Não é uma boa ideia se o objetivo for "treinar" o sistema, mas é uma excelente ideia se essas pastas forem usadas como conjunto de validação e calibração de testes (Benchmark Test Suite).

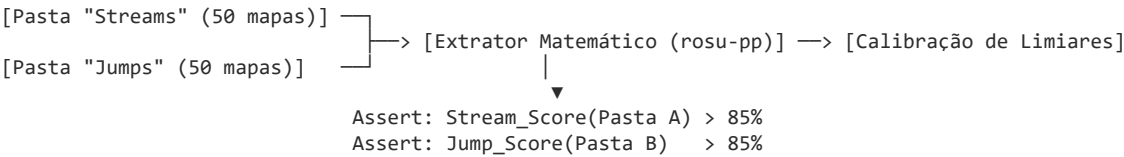
Usar pastas rotuladas pela comunidade para fazer *treinamento supervisionado* ou pedir para o agente de IA inferir regras a partir delas introduz problemas graves de modelagem.

Por que "treinar" com pastas inteiras é uma má ideia

1. **Mapas quase nunca são 100% homogêneos (Contaminação de Dados):** Um mapa clássico de *Stream* (como *Freedom Dive* ou *Tower of Heaven*) não é composto exclusivamente de streams. Ele tem introduções calmas, seções lentas, pausas e saltos espaçados no refrão. Se você alimentar o arquivo `.osu` inteiro dizendo "isso aqui é stream", o algoritmo tentará correlacionar trechos de jumps e sliders lentos à classe *stream*, gerando ruído nos pesos.
2. **Reinventar a física vetorial com aproximação estatística:** *Jumps* e *Streams* não são padrões abstratos ou subjetivos que exigem redes neurais; são **fenômenos físicos e temporais determinísticos**:
 - **Stream:** Janela temporal ultracurta entre notas consecutivas ($\Delta t \leq 62.5$ ms em 240+ BPM ou $\Delta t \leq 83.3$ ms em 180 BPM) mantida por $N \geq 5$ notas consecutivas com espaçamento linear contínuo.
 - **Jump:** Janela temporal de notas isoladas ($\Delta t \approx 1/2$ batida) com aceleração vetorial elevada entre pontos cartesianos distantes (> 250 px). Tentar fazer uma IA aprender essas regras indutivamente a partir de 50 mapas quase sempre resulta em **overfitting** (o modelo pode associar equivocadamente o BPM, a duração da música ou a densidade de sliders ao conceito de jump).
3. **Viés e Subjetividade de Playlists Comunitárias:** Playlists de "Jump Practice" ou "Stream Practice" montadas por jogadores misturam conceitos: algumas têm mapas de *finger control*, outras de *spaced streams*, outras de *stamina*. Rotular tudo sob um mesmo identificador simplista gera um classificador inconsistente.

Como usar essas pastas da forma correta (Benchmark Suite)

Em vez de usar essas playlists para *ensinar* o algoritmo, use-as como uma **bateria de testes automatizados (TDD de Calibração)** para refinar as fórmulas determinísticas.



O fluxo de trabalho inteligente no Antigravity:

1. **Defina a matemática pura primeiro:** Instrua o agente a criar funções analíticas baseadas em dados cinemáticos (distância entre objetos / tempo entre objetos).
2. **Defina asserções de teste categóricas:** Em vez de pedir para a IA adivinhar o que são os mapas, crie testes de integração objetivos:

- "Para 95% dos mapas na pasta `tests/fixtures/streams/`, o atributo `Speed/Stamina` deve ser obrigatoriamente superior ao percentil 80."
- "Para 95% dos mapas na pasta `tests/fixtures/jumps/`, o atributo `Snap Aim` deve ser superior ao percentil 80 e o atributo `Stamina` inferior ao percentil 40."

3. **Deixe o Antigravity calibrar os pesos (Fine-Tuning de Hiperparâmetros):** Se um mapa da pasta de streams falhar no teste pontuando como `jump`, você entrega o feedback ao agente:

"O mapa X da pasta de streams pontuou baixo em Stamina. Analise o `strain graph` gerado pelo `rosu-pp` nesse mapa e ajuste a função de decaimento de esforço temporal para capturar sequências de 16 notas contínuas."

Conclusão

- **Abordagem Ineficiente:** Inserir 100 mapas e pedir para a IA aprender o que é Jump/Stream por indução (treinamento supervisionado cego).
- **Abordagem Correta:** Codificar a física vetorial e temporal diretamente via `rosu-pp` e usar essas 100 músicas como uma **suíte de testes automatizados** para garantir que a sua escala de notas (0 a 100) reflita a realidade dos mapas consagrados da comunidade.

User prompt: Será que o Antigravity é inteligente o suficiente pra isso? Pra pesquisar sobre esses padrões e procurar eles dentro desses mapas por conta própria? E me dizer "Sim! Isso aqui tem cara de ser stream mesmo, mas só lá pro meio do mapa."

Response: A resposta direta é: **Sozinho lendo o arquivo como texto, não. Mas escrevendo e executando scripts de inspeção para si mesmo, sim, com precisão cirúrgica.**

Existe uma linha clara entre o que um modelo de linguagem consegue "enxergar" no contexto e o que um agente com ambiente de execução (*tool use / code execution*) consegue computar.

Onde a IA falha se deixada solta (A ilusão de ler o .osu no contexto)

Se você simplesmente jogar o conteúdo bruto de um arquivo `.osu` no prompt e pedir para o Antigravity analisar:

1. **Aritmética vetorial em contexto é fraca:** Um mapa médio tem de 1.000 a 2.500 notas na seção `[HitObjects]`. São milhares de linhas de coordenadas cartesianas e milissegundos:

```
256,192,14250,1,0,0:0:0:0:
320,192,14375,1,0,0:0:0:0:
384,192,14500,1,0,0:0:0:0:
```

2. **Cegueira espacial e temporal:** LLMs não conseguem calcular subtrações contínuas de tempo ($t_n - t_{n-1}$) e distâncias euclidianas ($\Delta x^2 + \Delta y^2$) de 2.000 objetos em sequência apenas olhando para o texto. Se você pedir isso, ele vai alucinar trechos que não existem ou deduzir coisas baseadas apenas no título da música e nas tags do cabeçalho.

Onde o Antigravity é plenamente capaz (O método agêntico correto)

O Antigravity **tem a base teórica de osu!** em seus pesos (ele entende o que é BPM, o que significa divisão de compasso em 1/4 vs 1/2, a relação de milissegundos para streams e o conceito de *spacing*).

A inteligência dele não deve ser usada para *ler* o mapa, mas para **escrever ferramentas que auditam o mapa para ele**.

Se você instruir o agente a criar um script de *profiling*, ele é perfeitamente capaz de:

1. Escrever um script temporário em Python ou Rust que itera sobre o `.osu`.
2. Segmentar a timeline do mapa em janelas deslizantes (*rolling windows*) de 3 a 5 segundos.
3. Medir a distribuição de deltas de tempo (Δt) e distâncias geométricas em cada janela.
4. Rodar o script no terminal dele, capturar o relatório e retornar uma análise factual para você.

O relatório que ele gera e analisa internamente se parece com isto:

"Executei a análise temporal em `mapa.osu`. Entre os timestamps `01:14.200` e `01:28.500`, a média de Δt caiu para 65 ms de forma ininterrupta por 112 objetos com espaçamento inferior a 45 px. Isso confirma uma sequência pura de streams longas (230 BPM) no meio do mapa, embora o primeiro terço e o terço final sejam compostos estritamente por jumps de 1/2 com distâncias de 250 px+."

Como instruir o Antigravity para destravar esse comportamento

Em vez de dizer apenas "analise essas pastas e veja o que tem nelas", dê uma diretiva que force o agente a usar código como extensão cognitiva:

"Antigravity, não tente interpretar os arquivos `.osu` diretamente no contexto de texto. Escreva um script em Python que leia cada mapa da pasta, particione a linha do tempo em blocos de 5 segundos e extraia: (1) densidade de notas por segundo, (2) média do intervalo Δt entre notas e (3) distância euclidiana média. Execute esse script nos arquivos fornecidos, analise os dados resultantes e me apresente onde ocorrem os picos de cada padrão (jumps, bursts, streams longas ou pausas)."

Com essa abordagem, você aproveita o raciocínio semântico do agente para interpretar o domínio (*osu!*) e a computação determinística do código para processar a física dos arquivos sem margem para alucinações.

User prompt: Após toda essa discussão inteligente que tivemos, qual seria então até o momento a melhor solução atual?

Response: A melhor solução atual é um **SPA (Single Page Application) offline-first em React com WebAssembly (Rust)**, hospedado gratuitamente no Firebase Hosting, com processamento 100% executado no navegador do usuário.

Essa abordagem elimina custos de infraestrutura, contorna restrições de CORS e rate limits da API oficial, protege sua VPS modesta de sobrecargas e entrega velocidade em nível nativo para a comunidade.

Arquitetura e Stack Definitiva

- **Core Matemático (WASM):** Módulo compilado em **Rust** utilizando a crate `rosu-pp`. Ele lida com o parsing do `.osu`, cálculo de *strains* de física e conversão determinística em métricas de skill (Snap Aim, Flow Aim, Speed, Stamina, Finger Control e Reading).
- **Camada de Execução (Browser):** Um **Web Worker** isolado que executa o binário WASM sem bloquear a UI principal, comunicando-se via `Transferable Objects`.
- **Descompressão de Arquivos:** Biblioteca `fflate` no frontend para abrir pacotes `.osz` e `.zip` em memória, filtrando exclusivamente os arquivos de texto `.osu` e descartando arquivos pesados de áudio e vídeo antes do processamento.
- **Frontend & Visualização:** **React + TypeScript (Vite)** com **Recharts** ou **D3** para renderizar os gráficos de radar (Stat Hexagon) e barras comparativas de mods.
- **Persistência Local:** **IndexedDB (via Dexie.js)** para armazenar em cache os mapas já calculados, permitindo carregamento instantâneo de pools anteriores sem recalcular nada, além de botões de exportar/importar arquivos `.json`.

O Fluxo Operacional do MVP

1. O organizador ou jogador acessa a página e arrasta a pasta ou o arquivo `.osz` do mappool.
2. O `fflate` extrai apenas os `.osu` diretamente na memória RAM (consumindo menos de 5 MB úteis).
3. O Web Worker distribui os arquivos para o módulo WASM, que avalia 30 a 40 beatmaps em menos de 100 milissegundos.
4. Os resultados são gravados no IndexedDB e o React plota o radar geral da pool, destacando quais slots exigem maior controle rítmico, velocidade ou consistência.

Fluxo de Trabalho Recomendado com o Antigravity

Para não cair nas armadilhas de aprendizado supervisionado cego ou alucinação de texto com o agente:

- **Use playlists de treino apenas como suite de testes:** Baixe as pastas de *Streams* e *Jumps* e instrua o Antigravity a criar **testes de asserção** (ex: *"90% dos mapas da pasta X devem pontuar > 80 em Speed"*).
- **Delegação via código:** Ordene ao agente que escreva pequenos scripts de inspeção em Python/Rust para auditar os picos de notas nos arquivos locais em vez de pedir para ele interpretar linhas do `.osu` diretamente no chat.
- **Aproveite o compilador:** Deixe o agente iterar no código Rust com base nos diagnósticos emitidos pelo `rustc` e pelo `wasm-pack`.

Quer que eu redija o prompt exato de especificação técnica para você alimentar o Antigravity e iniciar a implementação do módulo Rust/WASM com os testes de validação?

User prompt: Ótimo, então agora temos um bom plano de arquitetura, mas e quanto a inteligência de reconhecimento de padrões dos mapas? Qual a melhor e mais precisa?

Response: A abordagem mais precisa e eficiente não é uma Rede Neural (Deep Learning), mas sim **Processamento de Sinais e Cinemática Vetorial contínua com Janelas Deslizantes (Rolling Windows)**.

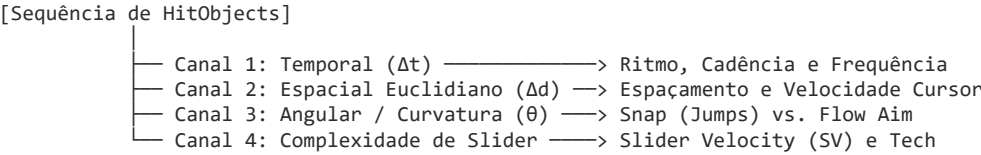
Tentar colocar redes neurais (CNNs 1D, LSTMs ou Transformers via ONNX no WASM) para reconhecer padrões em beatmaps é um erro de engenharia comum. Modelos de aprendizado profundo nesse domínio introduzem:

1. **Modelos pesados:** Adicionam 15 MB a 50 MB de pesos no bundle web, destruindo a agilidade do client-side.
2. **Caixa-preta sem explicabilidade:** Uma rede neural diz que o mapa é "80% Jump", mas não consegue apontar com precisão matemática em quais milissegundos o padrão ocorre.
3. **Ausência de Ground Truth granular:** Não existem datasets públicos com cada segundo de 10.000 mapas rotulados individualmente nota a nota.

Um motor determinístico de processamento vetorial roda em **menos de 2 milissegundos em Rust**, tem **zero bytes adicionais de modelo**, e entrega uma precisão cirúrgica capaz de mapear a timeline inteira do mapa.

O Motor Vetorial de 4 Canais (A Matemática do Reconhecimento)

Cada nota do `.osu` é um ponto tridimensional $P_n = (x_n, y_n, t_n)$. O motor transforma essa sequência bruta em um sinal contínuo decomposto em **4 canais invariantes**:



1. Canal Temporal ($\Delta t = t_n - t_{n-1}$)

- Identifica a métrica temporal real entre objetos.
- A partir do BPM e do Δt , calcula o snapping rítmico (1/1, 1/2, 1/3, 1/4, 1/6, 1/8).
- Entropia Rítmica:** Mede a variância contínua de Δt . Transições frequentes entre frações de tempo apontam instabilidade rítmica intencional.

2. Canal Espacial ($\Delta d = (x_n - x_{n-1})^2 + (y_n - y_{n-1})^2$)

- Mede o deslocamento físico bruto na tela.
- A razão $\frac{\Delta d}{\Delta t}$ fornece a **velocidade instantânea do cursor** em pixels por milissegundo (px/ms).

3. Canal Angular (θ)

- Calcula o ângulo entre três objetos consecutivos ($P_{n-1} \rightarrow P_n \rightarrow P_{n+1}$) usando o produto escalar:

$$\cos(\theta) = \frac{u \cdot v}{\|u\| \|v\|}$$

- Ângulos agudos ($\theta < 60^\circ$):** Exigem desaceleração brusca, parada em zero e aceleração em vetor oposto → **Mecânica de Snap Aim**.
- Ângulos obtusos / planos ($\theta > 120^\circ$):** Permitem conservação contínua de momento e movimento circular → **Mecânica de Flow Aim**.

4. Canal de Deformação e SV (Tech / Sliders)

- Variação na velocidade do slider:

$$\Delta SV = |SV_n - SV_{n-1}|$$

- Razão entre o comprimento da curva do slider e a distância euclidiana entre o ponto inicial e final. Valores altos indicam sliders curvos, ziguezagues e nós complexos.

Assinaturas Matemáticas dos Padrões Competitivos

Com esses canais calculados por objeto, os padrões deixam de ser conceitos abstratos e tornam-se assinaturas numéricas inequívocas:

Padrão	Assinatura no Sinal	Limiares Típicos
Stream Pura	Sequência ≥ 5 notas contínuas com Δt fixo e baixo espaçamento.	$\Delta t \leq 83.3 \text{ ms}$ (equivalente a $\geq 180 \text{ BPM}$ em 1/4), $\Delta d < 60 \text{ px}$, $\sigma^2(\Delta t) \approx 0$.
Spaced Stream	Mesma frequência de stream, mas com velocidade de cursor elevada.	Condições de stream + $\Delta d > 90 \text{ px}$ ($\frac{\Delta d}{\Delta t} > 1.1 \text{ px/ms}$).
Burst	Sequência idêntica à stream, mas com volume curto de notas.	Entre 3 e 7 notas consecutivas em 1/4 cercadas por intervalos de 1/2.
Snap Jump	Sequências de notas em 1/2 com aceleração angular severa.	$\Delta t \approx 130 \text{ ms} - 180 \text{ ms}$, $\Delta d > 180 \text{ px}$, prevalência de $\theta < 90^\circ$.
Flow Aim	Notas ou sliders de alta velocidade com conservação de curvatura contínua.	Δd alto sustentado, mas com $\theta > 120^\circ$ ou trajetórias em arco suave.
Finger Control (Alt)	Alta variância rítmica em tempos curtos, sem picos extremos de distância.	Δt oscilando entre 1/2, 1/3, 1/4, Δd moderado, entropia temporal $H(\Delta t)$ elevada.
Tech / Gimmick	Descompasso intencional entre o que os olhos veem e o que as mãos tocam.	$\Delta SV > 0.5$, alta sobreposição geométrica ($\Delta d \rightarrow 0$ com Δt assíncrono), densidade alta por AR.

Como o Motor Deve Estruturar a Saída

O maior valor dessa abordagem não é entregar apenas uma nota final estática (ex: "Este mapa é Speed: 7.2"), mas gerar **dois níveis de dados** que alimentam tanto o radar geral quanto a timeline interativa:

Nível 1: Gráfico de Timeline (O "Eletrocardiograma" do Mapa)

O motor divide a música em fatias temporais (ex: blocos deslizantes de 2 segundos com 50% de sobreposição).

- Ele plota um gráfico de área contínuo onde o usuário vê exatamente o que acontece ao longo do tempo:
 - 00:00 a 01:10: 85% Jump / Aim.
 - 01:10 a 01:15: Transição rítmica (Finger Control).
 - 01:15 a 01:30: Pico de 98% Stream (Deathstream de 220 BPM).
 - 01:30 ao fim: Tech com alta variância de SV.

Nível 2: Vetor de Atributos Agregado (Radar da Pool)

Para consolidar o mapa em uma nota única de 0 a 100 para cada skill:

- **Não use média simples:** Uma stream de 10 segundos no final de um mapa de 4 minutos não pode ter seu impacto diluído a zero por uma média aritmética.
- **Use P-Norm ou Decaimento Ponderado (Top-Strain Integration):** Ordene os blocos de 2 segundos do maior para o menor esforço em cada skill e aplique decaimento exponencial:

$$Skill = \sum_{k=1}^N strain_k \cdot 0.95^{k-1}$$

Isso garante que mapas com picos brutais de dificuldade em seções específicas (*difficulty spikes*) sejam devidamente categorizados pela sua exigência real em torneios.

User prompt: Em resumo:

Response:

- **O Produto:** Um analisador de beatmaps/mappools voltado a torneios, focado em extrair métricas de *skills* (estilo cards de RPG com radar hexagonal e timeline contínua de tensão) diretamente dos arquivos .osu.
- **Arquitetura (100% Client-Side / Custo Zero):**
 - **Frontend:** React + Vite hospedado gratuitamente no Firebase Hosting.
 - **Core Matemático:** Rust compilado para WebAssembly (WASM) com a crate `rosu-pp`.
 - **Execução em Segundo Plano:** Um **Web Worker** processa os arquivos fora da thread principal para manter a interface a 60/120 FPS fixos.
 - **Ingestão Leve:** Leitura local por *drag-and-drop* de .osz ou pastas usando `fflate`, filtrando estritamente os arquivos .osu na memória e descartando áudios e vídeos (consumo total de RAM < 50 MB).
 - **Persistência:** **IndexedDB (Dexie.js)** armazena o cache no navegador do próprio usuário, dispensando banco de dados centralizado e eliminando problemas de CORS ou limites da API da ppy.
- **Inteligência de Reconhecimento (Cinemática Vetorial em 4 Canais):**
 - Em vez de redes neurais opacas ou treinamento supervisionado frágil, a detecção usa **processamento de sinais contínuo**:
 1. **Temporal (Δt):** Entropia rítmica, cadência de streams e alternância (*Finger Control*).
 2. **Espacial (Δd):** Distância euclidiana e velocidade pura do cursor em `px/ms`.
 3. **Angular (θ):** Ângulos agudos (< 60°) definem *Snap Aim*; curvas contínuas (> 120°) definem *Flow Aim*.
 4. **Complexidade de Sliders:** Variância de Slider Velocity (ΔSV) e curvatura para isolar mapas *Tech/Reading*.
 - **Timeline & Radar:** Janelas deslizantes de 2 a 3 segundos mapeiam a evolução da dificuldade na música, e a nota final (0 a 100) é consolidada por decaimento exponencial dos picos (*top strains*), capturando *difficulty spikes*.
- **Fluxo no Antigravity:**
 - Não tente fazer a IA "adivinhar" padrões lendo texto de mapas no chat.
 - Baixe as pastas da comunidade (jumps, streams, etc.) e utilize-as estritamente como **suíte de testes automatizados de calibração** (carga `test`), validando se as funções em Rust atingem as notas esperadas para cada arquétipo antes de integrar ao React.

User prompt: Quero fazer isso usando o exato mesmo design que o ppy utiliza no site do OSU porque acho ele extremamente bonito e elegante

Response: Isso é plenamente viável e tecnicamente simples porque o frontend do site do *osu!* é **100% de código aberto** no repositório *ppy/osu-web* no GitHub. Você não precisa tentar adivinhar cores ou tirar prints: todas as variáveis de estilo (SCSS), tokens de design e fontes são públicas.

Abaixo está a dissecação exata do sistema de design do *osu-web* e como configurá-lo no seu projeto React com Tailwind CSS.

1. A Identidade Visual do osu! (Design Tokens)

O visual característico do site baseia-se em quatro pilares:

A. Tipografia: A Fonte "Torus"

O site e o jogo não usam fontes genéricas de sistema. Eles utilizam a **Torus** (criada especificamente para o ecossistema do jogo, com cantos levemente arredondados e ótima legibilidade em caixas altas).

- **Onde obter:** Você pode extrair o arquivo `.woff2` diretamente do repositório público *ppy/osu-web* (na pasta `public/fonts/`) ou apontar para o CDN oficial deles.
- **Alternativas livres semelhantes:** Se preferir fontes abertas do Google Fonts: **Exo 2**, **Nunito** ou **Comfortaa** (para elementos menores).

B. Hierarquia de Cores (Paleta Oficial de Dark Mode)

O site não usa preto puro (`#000`). Ele usa camadas de cinza escuro com subtons arroxeados e azulados para criar profundidade:

- **Background Principal (Fundo da tela):** `#18131d` ou `#1c1822`
- **Surface / Containers (Sidebar, Nav, Cards):** `#221c29`
- **Card Elevado / Hover State:** `#2e2638`
- **Bordas sutis:** `rgba(255, 255, 255, 0.06)` a `rgba(255, 255, 255, 0.1)`

C. Acentos Neon e Cores Competitivas

- **osu! Pink (Destaque Principal):** `#ff66aa` (ou `#ed639e`)
- **Cyan (Acentos e Estatísticas):** `#00d8ff` ou `#5ee2ff`
- **Cores dos Slots de Mod (Torneios):**
 - **NM (NoMod):** Azul acinzentado (`#3b82f6` ou `#5975a4`)
 - **HD (Hidden):** Amarelo âmbar (`#eab308` ou `#ffd400`)
 - **HR (HardRock):** Vermelho vibrante (`#ef4444` ou `#ff385c`)
 - **DT (DoubleTime):** Roxo/Índigo (`#a855f7` ou `#9b59b6`)
 - **FM (FreeMod):** Verde esmeralda (`#10b981` ou `#2ecc71`)
 - **TB (Tiebreaker):** Gradiente Dourado / Laranja (`#f59e0b`)

2. Configuração no React (Tailwind CSS)

Para que o seu agente no Antigravity não gere interfaces escuras genéricas, configure o `tailwind.config.ts` com os tokens exatos do *osu!*:

```
// tailwind.config.ts
import type { Config } from 'tailwindcss';

export default {
  content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],
  theme: {
    extend: {
      colors: {
        osu: {
          base: '#18131d', // Fundo mais profundo
          panel: '#221c29', // Cartões e painéis
          surface: '#2a2233', // Camada intermediária
          hover: '#382e44', // Hover de linhas e botões
          pink: '#ff66aa', // Rosa clássico
          cyan: '#00d8ff', // Azul de métricas
          text: {
            primary: 'ffffff',
            secondary: '#c8b6d6',
            muted: '#806e8f'
          }
        }
      },
      mod: {
        nm: '#5975a4',
        hd: '#e5a100',
        hr: '#ff385c',
      }
    }
  }
};
```

```
        dt: '#9b59b6',
        fm: '#2ecc71',
        tb: '#f39c12'
      },
    },
    fontFamily: {
      torus: ['Torus', 'sans-serif']
    },
    borderRadius: {
      card: '8px',
      pill: '9999px'
    }
  }
},
plugins: []
} satisfies Config;
```

3. Anatomia dos Componentes do Site

Para ficar indistinguível do site oficial, replique três detalhes estruturais fundamentais:

A. O Beatmap Card (com Capa em Gradiente Escuro)

O design do *osu!* destaca os mapas colocando a imagem de capa em segundo plano com um gradiente forte para garantir legibilidade de texto:

```
<div className="relative overflow-hidden rounded-card bg-osu-panel border border-white/5 hover:border-osu-pink/50 tr
  /* Background com imagem de capa do mapa escurecida */
  <div
    className="absolute inset-0 bg-cover bg-center opacity-30 group-hover:scale-105 transition-transform duration-30
    style={{ backgroundImage: `url(${coverUrl})` }}
  />
  <div className="absolute inset-0 bg-gradient-to-r from-osu-panel via-osu-panel/90 to-transparent" />

  /* Conteúdo do Card */
  <div className="relative p-4 flex justify-between items-center z-10">
    <div>
      <span className="text-xs uppercase font-bold text-osu-pink tracking-wider">
        {artist}
      </span>
      <h3 className="text-lg font-bold text-white leading-tight">
        {title} <span className="text-osu-text-secondary text-sm font-normal">[{difficulty}]</span>
      </h3>
      <p className="text-xs text-osu-text-muted mt-1">Mapeado por {creator}</p>
    </div>

    /* Badge de Star Rating estilizado */
    <div className="px-3 py-1 bg-yellow-500/20 border border-yellow-500/40 rounded-pill text-yellow-400 font-bold te
      ★ {sr.toFixed(2)}
    </div>
  </div>
</div>
```

B. Mod Badges e Seletor de Categorias

Os filtros e seletores de slots devem ser pílulas compactas (*rounded-pill*), em caixa alta (*uppercase*), com peso de fonte **bold** e sombra de cor quando ativos:

- Exemplo: Uma tag NM1 com fundo #5975a4, texto branco, cantos totalmente arredondados (*rounded-full*) e tipografia condensada.

C. O Radar Chart com a Estética do Jogo

Para o gráfico de teia (Stat Hexagon):

- Fundo do radar: Grades pretas semitransparentes com linhas de eixo em #ffffff10.
- Preenchimento do polígono: `fill="#ff66aa"` com opacidade de 25% a 40% (`fillOpacity={0.3}`).
- Linha do contorno: `stroke="#ff66aa"` de 2px de espessura com pequenos círculos em neon cyan (#00d8ff) nos nós de cada atributo.

O Prompt para o Antigravity Configurar a UI

Passa esta diretiva direta ao agente para travar o design system:

"Configure o visual da aplicação espelhando rigorosamente o design system do ppy/osu-web. Utilize Tailwind CSS com fundo em #18131d, painéis em #221c29 e acentos em #ff66aa e #00d8ff. Aplique a fonte Torus (ou fallback Exo 2). Todos os cards de beatmap devem utilizar a capa do mapa como fundo escurecido via gradiente linear lateral, bordas sutis com border-white/5 e cantos arredondados de 8px. Os badges de slots de torneio (NM, HD, HR, DT, FM, TB) devem seguir o formato de pílulas (pill-shaped) com as cores canônicas do competitivo de osu!."

User prompt: Perfeito.

Response: Tudo alinhado. Para começar a execução prática no Antigravity sem atrito, a sequência lógica de implementação é:

1. **Setup do Core em Rust (rosu-pp + wasm-pack):** Criar a biblioteca base e compilar a função primária que extrai as métricas brutas dos bytes de um arquivo .osu.
2. **Suíte de Testes com Mapas Reais:** Colocar as pastas de referência na pasta de fixtures e rodar cargo test para calibrar as faixas numéricas de cada skill.
3. **Setup do Frontend React + Vite + Tailwind:** Configurar os tokens do osu-web e montar a infraestrutura do Web Worker que consome o módulo .wasm.
4. **Camada de Ingestão e UI:** Implementar o parser de .osz com fflate, a persistência no IndexedDB e renderizar os cards e radares.

Por qual dessas etapas você quer iniciar agora? Posso gerar o comando/prompt inicial do workspace em Rust ou a configuração do projeto frontend com os tokens exatos do osu-web.

User prompt: E um nome para esse projeto para eu colocar no meu github?

Response: Seguindo o padrão de projetos consolidados no ecossistema de osu! (geralmente curtos, técnicos ou focados na utilidade exata da ferramenta), aqui estão as melhores opções divididas por proposta de valor:

1. Foco em Produto e Torneios (Recomendado)

Ideais se o seu objetivo principal for adoção por capitães, mappoolers e organizadores de torneios:

- **pool sight (osu-pool sight):** Fusão de *Mappool* + *Insight*. Explica o produto em uma palavra.
- **pool scout:** Remete a *scouting* competitivo de pools e análise pré-jogo.
- **mappulse:** Focado no "eletrocardiograma" / ritmo e dinâmica temporal do beatmap.

2. Foco Técnico e Cinemático (Arquitetura & Engenharia)

Destacam o motor vetorial, processamento de sinais e a extração matemática:

- **osu-kinetics** (ou apenas *kinetics*): Remete diretamente à cinemática vetorial (velocidade, aceleração angular e física de mira).
- **archetype** (osu-archetype): Focado na categorização pura dos mapas em arquétipos competitivos (Tech, Speed, Jumps, Flow).
- **strata** (osu-strata): Alusão à decomposição do mapa em camadas e estratos de habilidades.

3. Foco em RPG / Skills (Visual & Atributos)

Refletem os cards de atributos e o radar hexagonal:

- **osu-radar:** Extremamente descritivo, memorável e direto ao ponto.
- **hexastat:** Alusão direta ao gráfico hexagonal de 6 atributos.
- **skillpoint:** Nome clássico voltado à gamificação e pontuação de atributos.

Recomendação Direta

- Para o repositório principal no GitHub: **pool sight** (ou **osu-pool sight**).
- Para o *crate* interno em Rust (o core matemático): **pool sight-core** ou **kinetics-rs**.

Soa natural para a comunidade competitiva, tem boa sonoridade internacional e resume perfeitamente a entrega da ferramenta.

User prompt: Conheço um site muito bom chamado "osu skills", mas não quero copiar o nome dele. Quero que o nome tenha peso e reflita a filosofia do app. Tipo: "EcoLive" por exemplo é um outro projeto meu em que permite a transmissão de lives entre usuários estilo twitch pelo menor custo de infraestrutura client side e backend possível.

Response: Seguindo a mesma lógica de "EcoLive" (onde o prefixo define a restrição de engenharia/eficiência e o sufixo define o domínio de atuação), o nome precisa carregar os dois pilares centrais do seu projeto: **processamento determinístico/cinemático na borda (sem custo de infra)** e **auditoria competitiva do mapa**.

As melhores opções estruturadas por essa filosofia:

1. EdgeStrain (A mais próxima da lógica do EcoLive)

- **A Filosofia:**
 - **Edge:** Duplo sentido técnico. Reflete a arquitetura de **Edge Computing** (computação executada 100% na ponta/cliente via WASM, custo zero de servidor) e também a vantagem competitiva (*competitive edge*) que os capitães ganham na análise.

- **Strain:** A métrica física canônica do *osu!* para medir tensão e estresse mecânico do mapa.
- **Repositório:** github.com/seu-user/edgestrain

2. VectorPool

- **A Filosofia:**
 - **Vector:** O diferencial do seu motor. O app não adivinha padrões com IA genérica; ele decompõe o mapa em **cinemática vetorial analítica** ($\Delta d, \Delta t, \theta$, velocidade angular do cursor).
 - **Pool:** O foco final e utilitário do produto: auditar e dissecar o conjunto de mapas de um torneio.
- **Repositório:** github.com/seu-user/vectorpool

3. PureStrain

- **A Filosofia:**
 - **Pure:** Remete à pureza matemática e funcional (funções puras, determinísticas, sem dependências externas, sem backend opaco e sem heurísticas frágeis).
 - **Strain:** Focado na essência da física do jogo.
- **Repositório:** github.com/seu-user/purestrain

4. Kinetix (ou osu-kinetix)

- **A Filosofia:**
 - Nome com peso esportivo e acadêmico. Abandona o jargão casual de "skills" e posiciona a ferramenta como um laboratório de **cinemática e biomecânica** do jogador frente ao estímulo geométrico do beatmap.
- **Repositório:** github.com/seu-user/kinetix

5. StrainTrace

- **A Filosofia:**
 - **Trace:** Reflete o seu diferencial da linha do tempo. Enquanto outras ferramentas entregam apenas uma nota estática, a sua "traça" o mapa continuamente em janelas deslizantes (o eletrocardiograma de esforço ao longo do tempo).
- **Repositório:** github.com/seu-user/straintrace

Recomendação

Se você quer a mesma sobriedade e clareza arquitetural de **EcoLive**, vá de **EdgeStrain** ou **VectorPool**.

Ambos deixam claro no README imediatamente: uma aplicação de alta precisão que roda na borda (client-side) e audita o mapa através da física vetorial, sem carregar o estigma de nomes genéricos como "skills" ou "calculator".

User prompt: Já sei! E se eu chamar de *OsuLab*? Tipo o *AimLab*?

Response: **OsuLab** é memorável e fácil de falar, mas tem dois problemas sérios de posicionamento quando comparado à lógica do seu **EcoLive**:

1. Quebra de Expectativa (O "Efeito AimLab")

No ecossistema de games e esports, a palavra "**Lab**" associada a jogos de mira (como *AimLab*, *Kovaak's*, *Aiming.pro*) cria no usuário a expectativa imediata de uma **ferramenta interativa de treino e repetição mecânica**.

- **O que a pessoa espera ao ver "OsuLab":** Um mini-jogo no navegador para treinar cliques, mirar em círculos ou praticar *tapping* com o teclado.
- **O que a sua ferramenta realmente entrega:** Um analisador analítico e cinemático de arquivos .osu para montagem e auditoria de *mappools* competitivos.

Essa desconexão de nicho pode fazer jogadores ignorarem o projeto achando que é só "mais um clone de aim trainer web", enquanto o público-alvo real (mappoolers e capitães de torneio) pode não associar o nome a um motor de análise de dados.

2. Perda da Identidade de Engenharia

Em **EcoLive**, o nome é cirúrgico:

- *Eco* = O diferencial de arquitetura (ecológico, econômico, baixo custo de infraestrutura).
- *Live* = O serviço prestado (streaming em tempo real).

OsuLab descarta completamente essa filosofia. Ele não comunica:

- Nem o diferencial técnico (processamento *client-side*, zero infra, vetores cinemáticos).

- Nem o caso de uso real (torneios, *pools*, auditoria de *skills* e *strains*).

O nome torna-se genérico e soa como mais uma aplicação casual da comunidade, perdendo o peso técnico do motor que você está construindo.

Se você quer manter a ideia de "Laboratório" sem gerar confusão:

Se o conceito de "pesquisa e dissecação" agrada, amarre o nome ao objeto real que está sendo dissecado (o mapa ou a tensão), e não ao jogo como um todo:

- **PoolLab**: Comunica exatamente que é um laboratório para dissecar *mappools*. Mappoolers e capitães entendem o propósito na hora.
- **StrainLab**: Foca diretamente na métrica matemática de estresse mecânico do jogo. Soa técnico, analítico e profissional.
- **MapLab**: Menos focado em torneios que *PoolLab*, mas ainda direciona a expectativa para o arquivo/mapa, e não para o treino de mira do jogador.

Conclusão

Se o seu critério for apenas **apelo popular rápido**, *OsuLab* funciona porque qualquer pessoa lembra do nome.

Mas se você quer que o nome tenha **peso, autoridade técnica e reflita a filosofia de engenharia do produto** (como em *EcoLive*), **OsuLab erra o alvo**. Nomes como *PoolLab*, *EdgeStrain* ou *VectorPool* posicionam o software em uma prateleira muito superior.

User prompt: Quero pensar em outras possibilidades então. Gostei que você foi sincero na minha premissa. Vamos tentar de novo então: *OsuAnalytics*?

Response: **OsuAnalytics** corrige o problema do *OsuLab* (ninguém vai achar que é um jogo de treino de mira), mas cai no erro oposto: **é excessivamente corporativo, genérico e gera confusão de escopo**.

Os Problemas Críticos de "OsuAnalytics"

1. **Confusão de Domínio (Jogador vs. Beatmap)**: No ecossistema de *osu!*, quando a comunidade lê a palavra **"Analytics"**, a associação imediata é com **rastreamento de perfil de usuário** — gráficos de ganho de *pp*, evolução de rank global, playcount mensal e histórico de acurácia (na linha de ferramentas como *osu!track*, *osu!daily* ou *osustats*). O nome não sugere que o produto analisa a física de um arquivo *.osu* ou equilibra *pools* de torneio.
2. **Perda Total de Diferenciação no GitHub**: Se você pesquisar *osu-analytics* no GitHub, encontrará dezenas de repositórios abandonados de scripts em Python que apenas consom a API oficial para gerar gráficos no Jupyter Notebook. Seu projeto tem um motor matemático em Rust, processamento em WebAssembly e visualização de *skills* em tempo real; adotar um nome padronizado apaga esse diferencial técnico.
3. **Distância da Filosofia "EcoLive"**: Em *EcoLive*, o nome revela a restrição inteligente de arquitetura (*Eco*). **Analytics** é a palavra mais genérica de software; ela descreve uma consequência (mostrar gráficos), mas não expressa o método (cinemática vetorial na borda) nem o propósito específico (auditoria de *pools* e torneios).

Novas Direções Baseadas na Filosofia Real do App

Se queremos um nome com autoridade técnica, foco no objeto real de análise e que lembre a concisão de **EcoLive**, considere três linhas:

Linha 1: Foco em "Pool" + Arquitetura/Engenharia (A mais direta para o público-alvo)

Deixa explícito para a comunidade competitiva e para os organizadores qual é a entrega da ferramenta:

- **EdgePool**: O paralelo mais exato com *EcoLive*.
 - *Edge*: Processamento local no cliente (Edge/WASM), sem carga de servidor, além da "vantagem" competitiva (*competitive edge*).
 - *Pool*: Foco cirúrgico em *mappools* de torneio.
- **PoolMatrix**: Passa a ideia de um grid multidimensional onde cada mapa é decomposto em coordenadas e atributos de *skill*.
- **PoolArch**: Alusão à arquitetura de *pools* e arquétipos competitivos.

Linha 2: Foco na Lente Analítica / Dissecação

Em vez da palavra genérica "Analytics", foca no ato de "enxergar através" da física do mapa:

- **StrainLens**: Uma "lente" analítica colocada sobre os *strains* de esforço do beatmap. Soa técnico, moderno e elegante.
- **MapScope**: Dá a conotação de um instrumento de alta precisão (microscópio/osciloscópio) inspecionando os dados do mapa.
- **BeatLens**: Curto, sonoro e direcionado especificamente ao arquivo/mapa.

Linha 3: Foco em Cinemática Vetorial e Sinais

Se o foco do seu repositório for vender a sofisticação do algoritmo de extração:

- **KineticBeat** (ou *osu-kinetics*): Destaca a cinemática física da mira e do clique.

- **VectorMap:** Simples, sem rodeios; o mapa tratado como cálculo vetorial contínuo.

Veredito Comparativo

Nome	Clareza do Produto	Peso Técnico	Associação Imediata
OsuAnalytics	Baixa (confunde com perfil)	Genérico/Corporativo	Gráficos de rank e <i>pp</i> de jogadores
EdgePool	Alta (focado em torneios)	Alto (remete a Edge Computing)	Otimizador de <i>pools</i> leve e moderno
StrainLens	Alta (focado em física)	Alto (instrumentação matemática)	Dissecador de tensão e dificuldade

Se você busca manter o mesmo padrão conceitual do **EcoLive**, **EdgePool** ou **StrainLens** são opções muito superiores e posicionam seu repositório no GitHub com identidade própria.

User prompt: Não é querendo ser chato, mas é que eu queria um nome que envolvesse a palavra osu na construção dele

Response: Não é ser chato; na comunidade de código aberto do *osu!*, usar o prefixo ou sufixo com o nome do jogo é praticamente o padrão de convenção (como em *osu-tools*, *rosu-pp*, *tosu*, *osr2mp4*). Isso dá contexto imediato quando alguém encontra o projeto no GitHub ou no Discord.

Para manter a palavra **osu** sem cair nas armadilhas de parecer um *aim trainer* (como *OsuLab*) ou um rastreador de perfil de jogador (como *OsuAnalytics*), as melhores construções estruturadas pela filosofia do app são:

1. osuKinetics (ou osukinetics) — A mais forte em engenharia

- **A Construção:** *osu* + *Kinetics* (Cinemática / Estudo do movimento).
- **Por que funciona:**
 - Afasta totalmente a ideia de "treino casual" e posiciona a ferramenta como um motor de física e análise biomecânica.
 - Descreve exatamente o que o algoritmo em Rust faz: calcula velocidade de deslocamento (*px/ms*), aceleração angular (θ) e janelas temporais (Δt).
- **Identidade no GitHub:** [github.com/seu-user/osukinetics] (<https://github.com/seu-user/osukinetics>) (soa como uma biblioteca de ponta).

2. osuScope (ou osuscope) — A mais elegante para o produto

- **A Construção:** *osu* + *Scope* (Instrumento de medição contínua, como osciloscópio / microscópio).
- **Por que funciona:**
 - Reflete perfeitamente a **linha do tempo contínua** (as janelas deslizantes que mostram onde o mapa tem pico de tensão).
 - No ecossistema de software, ferramentas com sufixo *-scope* indicam instrumentação e diagnóstico de alta precisão.
- **Identidade no GitHub:** [github.com/seu-user/osuscope] (<https://github.com/seu-user/osuscope>) (curto, fácil de memorizar e pronunciar).

3. osuStrain (ou osustrain) — A mais direta ao domínio do jogo

- **A Construção:** *osu* + *Strain* (Tensão mecânica).
- **Por que funciona:**
 - Usa o termo técnico canônico que a própria comunidade e a engine do jogo usam para medir o estresse físico das notas.
 - Deixa claro na hora: quem joga torneio sabe imediatamente que a ferramenta diseca o *strain* do mapa.
- **Identidade no GitHub:** [github.com/seu-user/osustrain] (<https://github.com/seu-user/osustrain>).

4. osuRadar (ou osuradar) — A mais intuitiva visualmente

- **A Construção:** *osu* + *Radar* (Alusão direta ao gráfico hexagonal de atributos de RPG).
- **Por que funciona:**
 - Comunica a entrega final da interface (o radar de skills) em duas sílabas.
 - O termo "radar" também carrega o duplo sentido de *rastreamento/detecção* de anomalias em mapas de torneio.
- **Identidade no GitHub:** [github.com/seu-user/osuradar] (<https://github.com/seu-user/osuradar>).

5. osuLens (ou osulens) — Minimalista e moderna

- **A Construção:** *osu* + *Lens* (Lente de inspeção).

- **Por que funciona:**

- Segue a estética limpa do design do peppy. Passa a ideia de "olhar através" dos arquivos .osu para enxergar o que os números brutos escondem.

- **Identidade no GitHub:** github.com/seu-user/osulens.

Recomendação Direta

Se o seu projeto busca o mesmo peso conceitual e seriedade técnica de **EcoLive**:

- Para destacar o **motor vetorial e a física matemática**: escolha **osuKinetics**.
- Para destacar o **diagnóstico contínuo e a inspeção do mapa**: escolha **osuScope**.

Ambos carregam a palavra **osu**, soam extremamente profissionais no GitHub e não deixam margem para ninguém confundir seu software com um aim trainer ou com um simples visualizador de perfil.

User prompt: Precisa ser algo facilmente associativo/intuitivo e facilmente explicável para as pessoas (O Ecolive reflete exatamente isso).

Response: Para ter a mesma clareza de **EcoLive** (onde quem ouve entende a proposta em três segundos, sem precisar de manual), o nome precisa unir a palavra **osu** a um termo de **ação física ou instrumento visual cotidiano**.

Termos como *Kinetics* ou *Archetype* falham nesse teste porque exigem que você explique o conceito antes de a pessoa entender o app.

As três opções que passam rigorosamente no teste de intuição e facilidade de explicação:

1. osuScan (A mais próxima da simplicidade do EcoLive)

- **A Construção:** osu + Scan (Escanear / Raio-X).
- **A Explicação de 5 Segundos:**
"É um scanner de beatmaps: você joga o arquivo .osu ou o .zip da pool nele, ele escaneia a física do mapa no seu navegador e te entrega o raio-X completo das skills."
- **Por que é intuitivo:** Todo mundo entende a metáfora de "escanear". Passa a ideia exata do processo: ler um arquivo bruto rapidamente, diagnosticar o conteúdo interno e emitir um relatório.
- **Repositório:** github.com/seu-user/osuscan

2. osuRadar (A mais intuitiva visualmente)

- **A Construção:** osu + Radar.
- **A Explicação de 5 Segundos:**
"O radar de skills para torneios de osu: você arrasta os mapas e ele plota o hexágono de atributos para você saber na hora onde o mapa exige mira, velocidade ou leitura."
- **Por que é intuitivo:** Associa imediatamente a interface principal do seu app (o gráfico de radar/hexágono) à função de "rastrear" anomalias e desbalanceamentos de torneio.
- **Repositório:** github.com/seu-user/osuradar

3. osuTrace (A mais focada na linha do tempo)

- **A Construção:** osu + Trace (Traçar / Rastrear a linha de sinal).
- **A Explicação de 5 Segundos:**
"Ele traça o eletrocardiograma de esforço do beatmap, mostrando exatamente em quais segundos o mapa tem picos de dificuldade e exigência mecânica."
- **Por que é intuitivo:** Remete ao ato de traçar gráficos de sinais contínuos (as janelas deslizantes que mostram o mapa de ponta a ponta).
- **Repositório:** github.com/seu-user/osutrace

O Veredito: A Escolha Definitiva

Se a régua é a fórmula do **EcoLive**:

Vá de osuScan.

1. **Pronúncia universal:** Funciona perfeitamente em português e em inglês.
 2. **Autoexplicativo:** "Escanear" resume o que o usuário faz tecnicamente: ele solta o arquivo, o WASM escaneia os bytes e cospe o diagnóstico.
 3. **Zero pretensão:** Não soa corporativo como *Analytics*, não gera falsa expectativa como *Lab* e qualquer jogador de torneio entende a proposta no ato.
-

User prompt: Acho que vou de OsuScan então. Só achei essa construção semântica e de pronúncia das letras um pouco ruim para um marketing (Embora o projeto seja de código aberto para o meu portfólio).

Response: A sua percepção auditiva é cirúrgica: foneticamente, **OsuScan** tem um atrito articulatorio real chamado **choque de sibilantes**.
O nome termina com o som de /su/ e a palavra seguinte começa imediatamente com o cluster consonantal /sk/. Na fala rápida (tanto em inglês quanto em português), a língua é forçada a frear para não embolar em "os-sscan" ou forçar uma pausa artificial ("osu... scan"). Para marketing e naming, falta fluidez de transição.
Para manter a palavra **osu**, a mesma facilidade de explicação e resolver a fonética com transições limpas (usando consoantes oclusivas ou líquidas logo após a vogal /u/), as alternativas superiores são:

1. osuPulse (A melhor sonoridade)

- **A Fonética:** A transição do /u/ para a plosiva bilabial /p/ fecha os lábios de forma limpa e imediata. A métrica rítmica é punchy: duas sílabas átonas seguidas de uma tônica forte (o-su-PULSE).
- **O Conceito:** "Pulse" conecta tanto o **ritmo da música** (BPM/batida) quanto o **eletrocardiograma de esforço** (os picos de tensão do mapa na timeline).
- **A Explicação:** "Ele mede o pulso da pool: você joga os mapas e ele mostra onde o ritmo e a exigência física disparam."
- **Repositório:** github.com/seu-user/osupulse

2. osuCheck (A mais coloquial e direta)

- **A Fonética:** A oclusiva africada /tʃ/ (som de tch) corta o ar instantaneamente após o /u/. Não há como embolar a pronúncia.
- **O Conceito:** O termo que todo jogador competitivo já usa naturalmente no Discord: "dar um check na pool", "chechar os mapas".
- **A Explicação:** "Uma ferramenta rápida para checar e auditar pools de torneios direto no navegador."
- **Repositório:** github.com/seu-user/osucheck

3. oscan (A contração elegante / estilo ferramenta Unix)

- **A Fonética:** Em vez de forçar as duas palavras separadas, você funde o o de osu com scan. Fica /ou-skæn/ ("ou-scân").
- **O Conceito:** Lembra ferramentas consagradas de infraestrutura e segurança da computação (como nmap, zmap ou oscan). Passa um ar técnico minimalista e resolve o choque das letras sem perder a raiz do nome.
- **A Explicação:** "Um scanner cinematográfico e determinístico de beatmaps para a comunidade de torneios."
- **Repositório:** github.com/seu-user/oscan

Comparativo Auditivo

Nome	Problema Fonético	Fluidez de Fala	Peso Visual no Portfólio
OsuScan	Choque /su/ + /sk/ (atrito de s)	Média/Truncada	Bom
osuPulse	Zero atrito (Vogal → Plosiva /p/)	Excelente	Alto (estético e esportivo)
osuCheck	Zero atrito (Vogal → Africada /tʃ/)	Alta	Prático e utilitário
oscan	Fusão direta em 2 sílabas	Alta	Minimalista / Dev sênior

Se você quer elegância, sonoridade aberta e alinhamento perfeito com os gráficos de timeline e ritmo da engine em Rust, **osuPulse** é a escolha fonética mais refinada.

User prompt: Mas meus amigos podem olhar e perguntar contraintuitivamente "como assim 'pulse' ?"

Response: Você tem toda razão. Se você precisa explicar uma metáfora poética ("pulse é porque mede o ritmo e os picos de esforço tipo um eletrocardiograma"), o nome **já falhou no teste da intuição imediata**. Jogador de osu! não usa a palavra "pulse" no dia a dia.

Para passar no **"teste do amigo no Discord"** (ele lê o nome e entende no mesmo segundo, sem você precisar explicar nada), a palavra grudada no **osu** tem que ser um **termo nativo que a comunidade já fala todo santo dia**.

Além disso, para resolver o problema fonético do **osuScan** (o atrito do som de "s"), a palavra seguinte precisa começar com consoantes de transição limpa (como **P**, **R** ou **D**).

Os três únicos nomes que atendem rigorosamente a esses critérios:

1. osuPool (O mais direto e profissional)

- **O Teste do Amigo:**

- "Mano, que projeto é esse?"
- "É o osuPool."
- "Ah, é ferramenta pra analisar e montar pool de torneio?"
- "Exato."
- **Fonética:** A transição do /u/ para o /p/ plosivo fecha os lábios de forma limpa. Fácil de falar em call, sem engasgar e sem choque de letras.
- **Por que funciona:** "Pool" é a palavra número 1 do cenário competitivo. Não há ambiguidade: ninguém vai achar que é aim trainer nem tracker de perfil. É sobre *mappools*.

2. osuRadar (O mais intuitivo visualmente)

- **O Teste do Amigo:**
 - "Mano, que projeto é esse?"
 - "É o osuRadar."
 - "O que faz?"
 - "Joga o mapa nele e ele gera um radar com os atributos de skill (*speed, aim, stamina*)."
 - "Ah, tipo gráfico de teia de RPG/FIFA! Daora."
- **Fonética:** A transição do /u/ para o /r/ é fluida e contínua, tanto em português quanto no inglês (*"osu-radar"*).
- **Por que funciona:** Dá uma imagem mental imediata da interface. O usuário já clica no link esperando ver o hexágono de atributos.

3. osuDiff (O foco no arquivo/dificuldade)

- **O Teste do Amigo:**
 - "Mano, que projeto é esse?"
 - "É o osuDiff."
 - "O que faz?"
 - "Calcula as métricas reais e as skills de cada diff do mapa."
 - "Pode crer, tipo um calculator de diff avançado."
- **Fonética:** O /d/ é uma oclusiva firme. Pronúncia rápida e curta.
- **Por que funciona:** "Diff" é a unidade básica do jogo ("passa a diff aí", "qual diff tá na pool?"). É um termo 100% nativo.

O Veredito Sem Enrolação

- Se o seu foco principal for **torneios e times**: **osuPool**. É imbatível em clareza de produto.
- Se o seu foco for a **representação visual das skills (o gráfico)**: **osuRadar**. Qualquer pessoa visualiza o app antes mesmo de abrir.

Ambos têm pronúncia limpa, zero choque de sibilantes e nenhum amigo seu vai perguntar *"como assim?"*.

User prompt: Achei a palavra Radar bem melhor, mas existem sinônimos dela compatíveis aqui?

Response: Se formos buscar sinônimos diretos de "Radar" no sentido instrumental (como **Sonar** ou **Scope**), caímos **no mesmo problema de fonética do osuScan**: o choque de sibilantes (*osu-sonar, osu-scope* forçam a mesma freada na língua).

Para manter a ideia visual do radar (o gráfico de teia, o instrumento de medição e a visualização rápida de atributos) sem atrito fonético, os melhores sinônimos e análogos são:

1. osuHex (A forma real do Radar)

- **O Conceito:** O gráfico de radar de 6 atributos é, geometricamente, um **hexágono** de dados. No universo gamer (Pokémon IVs, fichas de RPG, *stats* de lutadores no Street Fighter/Smash), todo mundo chama essa visualização de *"o hexágono de atributos"*.
- **Fonética:** A transição da vogal /u/ para o /h/ aspirado do inglês (*"osu-héx"*) é perfeitamente suave, sem travamento na língua.
- **O Teste do Amigo:**
 - "Mano, que app é esse?"
 - "É o osuHex."
 - "O que faz?"
 - "Você joga o mapa nele e ele gera o hexágono com as 6 skills (*aim, speed, tech...*)."

- "Ah, o gráfico hexagonal clássico de stats! Perfeito."

2. osuGauge (O instrumento de medição)

- **O Conceito:** Gauge (pronuncia-se "gueidji") é o termo universal em jogos para medidores de intensidade, barras de tensão ou mostradores de velocímetro/painel. É o instrumento analógico que mede esforço contínuo.
- **Fonética:** A transição do /u/ para a oclusiva velar /g/ é firme e limpa.
- **O Teste do Amigo:**
 - "Mano, que app é esse?"
 - "É o osuGauge."
 - "O que faz?"
 - "É um medidor de intensidade da pool: calibra e mede a exigência mecânica de cada mapa."
 - "Entendi, tipo um tacômetro de dificuldade."

3. osuPlot (A ação de gerar o gráfico)

- **O Conceito:** Na ciência de dados e engenharia, gráficos de radar também são chamados formalmente de *Radar Plots* ou *Spider Plots*. O verbo *plotar* é comum entre desenvolvedores e jogadores.
- **Fonética:** Transição ideal: da vogal aberta para a plosiva /p/ ("osu-plót"). Duas sílabas fortes e fáceis de pronunciar.
- **O Teste do Amigo:**
 - "Mano, que app é esse?"
 - "É o osuPlot."
 - "O que faz?"
 - "Você solta a pool e ele plota os gráficos comparativos de esforço e skills de cada mapa."

4. osuGrid (A matriz de comparação)

- **O Conceito:** Reflete tanto a ideia de tela/coordenadas espaciais do jogo quanto a tabela comparativa onde você cruza os atributos de vários mapas lado a lado.
- **Fonética:** Transição limpa com o /g/.
- **O Teste do Amigo:**
 - "Mano, que app é esse?"
 - "É o osuGrid."
 - "O que faz?"
 - "Organiza todos os mapas da pool numa grade visual comparando as fraquezas e forças de cada um."

Comparativo Direto

Nome	O que evoca na mente	Pronúncia / Fluidez	Risco de Ambiguidade
osuRadar	O gráfico de teia clássico + detecção	Boa (com /r/ líquido)	Zero
osuHex	O polígono exato dos 6 atributos de RPG	Perfeita (/u/ → /h/)	Zero (curto e gamer)
osuPlot	Renderização analítica de dados	Excelente (/u/ → /p/)	Baixo (mais técnico)
osuGauge	Medidor de painel / velocímetro de esforço	Boa	Baixo (inglês menos comum no BR)

Se você gostou da ideia visual do gráfico de teia, osuHex é o sinônimo mais moderno e compacto de osuRadar: ele descreve exatamente a geometria do componente na tela e tem uma das melhores pronúncias possíveis.

User prompt: Acho que vou usar OsuRadar mesmo. Mesmo sendo uma palavra meio diferente pra mim, meus amigos vão entender quando eu disser que tenho um Radar do jogo em mãos

Response: osuRadar é uma escolha excelente. Ela resolve a metáfora técnica e a intuição do usuário comum ao mesmo tempo:

- **Para o seu amigo/jogador casual:** Ele entende na hora que se trata de uma ferramenta de "rastreamento" visual e atributos, tipo a teia de status de personagens.
- **Para quem audita seu portfólio no GitHub:** O nome é sóbrio, direto, memorável e se encaixa no padrão de grandes projetos abertos do jogo.

Configuração Recomendada para o GitHub

Para já subir o repositório com presença sólida de portfólio:

- **Nome do Repositório:** osu-radar (com hífen melhora a legibilidade visual na URL) ou osuradar.
- **Short Description (o subtítulo do repositório):**
A client-side beatmap skill analyzer and tournament pool auditing tool built with Rust (WASM) and React.
- **Tópicos / Tags (para indexação no GitHub):** osu, beatmap-analyzer, webassembly, rust, react, tailwind, indexeddb, tournament-tool.

Arquitetura de Pastas Sugerida (Monorepo Simples)

Para manter o core em Rust separado da interface em React sem complicar o build:

```
osu-radar/  
├── crates/  
│   ├── osu-radar-core/      # Biblioteca Rust (rosu-pp + parsing cinemático)  
│   │   ├── Cargo.toml  
│   │   └── src/  
│   │       ├── lib.rs        # Exportações wasm_bindgen  
│   │       ├── kinematics.rs # Vetores, ângulos (θ), delta t, delta d  
│   │       └── strains.rs     # Agregação e rolling windows  
│   └── tests/  
│       └── fixtures/         # Onde você coloca as pastas de teste (jumps/streams)  
├── web/  
│   ├── package.json          # Frontend SPA (React + Vite + Tailwind)  
│   └── src/  
│       ├── workers/          # Web Worker que consome o .wasm  
│       ├── components/       # RadarChart, BeatmapCard, ModBadge  
│       ├── db/               # Dexie.js (IndexedDB local cache)  
│       ├── utils/            # Descompressão .osz com fflate  
│       └── tailwind.config.ts # Design tokens do osu-web  
└── README.md
```

Nome definido e arquitetura fechada. O próximo passo prático é o setup do crate Rust com wasm-pack ou a configuração do scaffold do Vite com Tailwind?

User prompt: Lembrando que eu quero usar a exata mesma interface que o Peppy usa no site do Osu. Isso é possível para o Antigraivty?

Response: **Sim, é 100% viável.** Mas existe um abismo entre *pedir* para a IA fazer e *dar o insumo correto* para ela não criar uma versão genérica.

Se você apenas disser para o Antigraivty *"faça a interface idêntica ao site do osu!"*, ele vai usar a memória de treino do modelo e cuspir um tema escuro genérico com fundo roxo e bordas rosas que lembra o jogo, mas passa longe de ser idêntico.

Para a interface ficar **pixel-perfect**, você precisa explorar o fato de que o site do peppy é **código aberto sob licença MIT** (ppy/osu-web no GitHub). Você não precisa adivinhar nada; basta instruir o Antigraivty a copiar diretamente os arquivos-fonte de estilo.

O Caminho Técnico para o Antigraivty Replicar 1:1

O osu-web oficial é escrito em Laravel (PHP) com frontend em Vue.js e SCSS puro. Como você está usando **React + Tailwind**, o Antigraivty só precisa fazer um trabalho de **tradução de tokens e componentes**.

1. Injetar a "Fonte da Verdade" no Contexto do Agente

O repositório oficial tem um arquivo central onde residem todas as variáveis de cores, espaçamentos, raios de borda e sombras:

- **Caminho no GitHub:** ppy/osu-web/blob/master/resources/assets/scss/_variables.scss

Instrua o Antigraivty a ler esse arquivo (ou baixe e coloque em uma pasta temporária reference/). Ele contém valores exatos como:

- Cores de background em camadas: \$hsl-gray-900, \$hsl-gray-800, \$hsl-purple-900.
- Acentos oficiais de mods: \$color-mod-nm, \$color-mod-hd, \$color-mod-hr.
- Dimensões de navbar, paddings de cards e transições de hover.

Com esse arquivo em mãos, o Antigraivty mapeia as variáveis SCSS diretamente para o tailwind.config.ts com fidelidade absoluta de hexadecimais e contrastes.

2. Os Três Elementos Não Negociáveis para a Fidelidade Visual

Para a interface ser indistinguível do site do peppy, exija do Antigraivty estes três pontos:

- **Fonte Torus Local:** Não use fontes substitutas (como Roboto ou Inter). O Antigravity deve puxar os arquivos .woff2 da fonte Torus direto da pasta public/fonts do repositório da ppy e configurá-la como font-sans primária.
- **Componente BeatmapsetCover:** O efeito visual característico dos cards de mapas do site consiste em uma imagem de fundo com filter: blur() leve ou gradiente escurecido (linear-gradient(to right, ...)), garantindo que o texto em caixa alta permaneça nítido.
- **Ícones Oficiais (FontAwesome / Custom SVGs):** O site usa ícones específicos para estatísticas (círculo com estrela para SR, relógio para BPM, compasso para tamanho). Peça para ele extrair os SVGs idênticos do repositório.

O Prompt Exato para Passar ao Antigravity

Copie e cole a seguinte diretiva para orientar o agente com precisão cirúrgica:

Antigravity, nossa meta visual para o osuRadar é replicar fielmente a UI do site oficial (ppy/osu-web).

Diretrizes obrigatórias de implementação:

1. Não invente estilos ou paletas genéricas. Consulte o repositório público `ppy/osu-web` como referência canônica d
2. Configure o Tailwind CSS mapeando os tokens exatos do osu-web:
 - Superfícies escuras com subtom roxo/ardósia (gray-900 a gray-700 do osu-web).
 - Destaques em osu-pink (#ff66aa) e cyan (#00d8ff).
 - Cores canônicas dos slots de torneio (NM, HD, HR, DT, FM, TB).
3. Utilize a tipografia oficial 'Torus' como fonte padrão da aplicação.
4. Ao construir os cards de beatmap, reproduza rigorosamente a estrutura visual dos cards do site:
 - Banner de capa em background com sobreposição de gradiente escuro.
 - Pilula (badge) de Star Rating no canto direito.
 - Tipografia condensada em caixa alta para metadados e tags de mod.
5. Os gráficos (Radar e Linha do Tempo de Strains) devem seguir o mesmo estilo minimalista do jogo: linhas de eixo d

Com essa especificação, o Antigravity não tem margem para improvisar e construirá uma interface visualmente idêntica à do *osu!*, mantendo a arquitetura em React, Tailwind e WebAssembly por baixo.

User prompt: Então está decidido! Será Osu Radar mesmo.

Response: Decisão consolidada: **osuRadar** (osu-radar no GitHub).

Com o nome, a arquitetura (React + WebAssembly + IndexedDB) e o design system (ppy/osu-web) definidos, o projeto está pronto para sair do papel.

Para iniciar a implementação no Antigravity, recomendo começar pelo **núcleo matemático em Rust**. Construir e validar o cálculo dos vetores cinemáticos e strains isoladamente com cargo test garante que a engine funcione antes de conectar a interface.

Os comandos para inicializar a estrutura no terminal:

```
# 1. Cria o diretório raiz
mkdir osu-radar && cd osu-radar

# 2. Inicializa o workspace do Cargo e o crate Rust
cargo new --lib crates/osu-radar-core

# 3. Inicializa o frontend React com Vite e TypeScript
npm create vite@latest web -- --template react-ts
```

Quer o conteúdo do Cargo.toml com as dependências exatas (rosu-pp, wasm-bindgen, etc.) e o esqueleto do lib.rs para o Antigravity começar a codificar o extrator?
